BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

A Data Driven Toggling Gain Complementary Filtering Approach for Orientation Estimation

Samnun Azfar

200041130

Ramisa Zaman Audhi

200041131

Mir Md Inzamam

200041240

Department of Computer Science and Engineering

Islamic University of Technology

September, 2025

A Data Driven Toggling Gain Complementary Filtering Approach for Orientation Estimation

Samnun Azfar
200041130
Ramisa Zaman Audhi
200041131
Mir Md Inzamam
200041240

Department of Computer Science and Engineering
Islamic University of Technology
September, 2025

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by **Samnun Azfar**, **Ramisa Zaman Audhi**, and **Mir Md Inzamam** under the supervision of **Dr. Abu Raihan Mostafa Kamal**, Professor, Department of Computer Science and Engineering and co-supervision of **Mohammad Ishrak Abedin**, Lecturer, Department of Computer Science and Engineering, Islamic University of Technology, Dhaka, Bangladesh. It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Dr. Abu Raihan Mostafa Kamal
Professor
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Date: September 29, 2025

Mohammad Ishrak Abedin
Lecturer
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Date: September 29, 2025

Samnun Azfar
Student ID: 200041130
Date: September 29, 2025

Ramisa Zaman Audhi
Student ID: 200041131
Date: September 29, 2025

Mir Md Inzamam
Student ID: 200041240
Date: September 29, 2025

Dedicated to the cats of IUT

Contents

List of Figures

List of Tables

List of Abbreviations

MARG	Magnetic Angular-Rate Gravity
INS	Inertial Navigation System
CF	Complemetary Filtering/Complemetary Fusion
EKF	Extended Kalman Filter
ML	Machine Learning
BROAD	Berlin Robust Orientation Assessment Dataset
RANSAC	RANdom SAmple Consensus
IMU	Inertial Measurement Unit
MEMS	Micro-Electro-Mechanical Systems
AHRS	Attitude and Heading Reference System

Abstract

One of the goals of an Inertial Navigation System (INS) is to estimate the 3D orientation of a body under motion based on given accelerometer, magnetometer and angular rate gyroscope readings. Complementary fusion is one of the most robust, mathematically simple and fast algorithms for fusing the accelerometer, magnetometer and angular rate gyroscope readings to estimate the 3D orientation. But complementary fusion of sensor data suffers from linear accelerations and constant gain problems. This thesis aims to solve the aforementioned issues in a data driven approach. In this work, we introduce two contributions for robust orientation estimation using raw inertial and magnetic sensor data. First, we propose a tree-based Extreme Gradient Boosting (XGB) model that effectively denoises raw gyroscope, accelerometer, and magnetometer signals. Second, we present a tree-based movement detection model that dynamically adapts fusion process through α-toggling, enabling improved robustness to linear accelerations and magnetic interference. Together, these contributions establish a data-driven fusion framework that enhances reliability beyond traditional fixed-gain filters. The proposed approach is validated on a real-world dataset and further evaluated using a custom-built mechanical turntable, providing controlled testing conditions for systematic performance assessment.

Chapter 1

Introduction

1.1 Inertial Navigation

Inertial Navigation refers to the process of determining the *position*, *velocity*, and *orientation* of an object relative to its initial navigation parameters, without relying on any externally located position-fixing devices [39]. Any system that estimates these quantities using inertial sensing is referred to as an Inertial Navigation System (INS).

An INS offers continuous and self-contained navigation capability, making it highly valuable in environments where external references such as GPS are unavailable, unreliable, or intentionally denied. This includes underwater operations, subterranean exploration, and urban canyons where signal blockage is common.

However, the performance of INS is significantly limited by inherent challenges. Since the system operates by integrating motion information over time, even small measurement errors accumulate, leading to a gradual but inevitable degradation of accuracy. This phenomenon, commonly referred to as *drift*, causes the estimates of position, velocity, and orientation to diverge from the true values. Low-cost implementations are especially vulnerable, as their sensors are prone to noise, bias instability, and other disturbances that exacerbate long-term error growth. These limitations make error mitigation and robustness central concerns in the design of practical INS solutions.

1.2 Inertial Navigation and MARG Sensors

To address the challenges of error accumulation in Inertial Navigation, modern systems often employ multiple sensors to jointly estimate position, velocity, and orientation. Two sensors are common to almost all such systems: the accelerometer, which measures the direction of the gravitational acceleration vector, and the gyroscope, which measures angular velocity. Additionally, some systems incorporate a magnetometer to sense the direction of magnetic north [35]. Together, these three sensors form the **MARG** (**M**agnetometer **A**ccelerometer **R**ate-**G**yro) sensor suite.

Each of these sensors contributes unique information, but also comes with inherent limitations. Gyroscopes provide accurate short-term orientation changes, but suffer from drift over time. Accelerometers, on the other hand, offer a stable long-term reference by sensing gravity, yet are easily corrupted by linear accelerations, making it difficult to separate gravitational effects from translational motion. Magnetometers provide an absolute reference for heading estimation relative to magnetic north, but are highly susceptible to distortions caused by nearby ferromagnetic materials and electromagnetic interference.

By fusing the outputs of these sensors, a MARG-based system can leverage their complementary strengths while compensating for individual weaknesses. For instance, gyroscopes can smooth out the short-term noise of accelerometers and magnetometers, while accelerometers and magnetometers can correct the long-term drift of gyroscopes. This sensor fusion approach has made MARG systems particularly attractive for low-cost, portable, and real-time orientation and navigation applications.

1.3 Orientation Estimation and MARG Sensors

Orientation estimation is one of the three fundamental challenges (*position*,*velocity*, and *orientation estimation*) that an Inertial Navigation System (INS) must address. The orientation of a 3D object can be mathematically expressed using Euler angles, rotation matrices, or quaternions [41]. An orientation estimation algorithm processes the raw measurements from MARG sensors and produces an orientation estimate in one of these representations.

As discussed in Section 1.2, individual MARG sensors suffer from inherent limitations such as drift, noise, linear acceleration sensitivity, and magnetic disturbances. To overcome these issues, orientation estimation relies on sensor fusion filters that combine the complementary strengths of the sensors while compensating for their weaknesses. Standard algorithms such as the Madgwick filter [29], Mahony filter [30], Extended Kalman Filter (EKF) [31], Algebraic Quaternion Algorithm (AQUA) filter [43] and Complementary filter [22] are widely used in practice. Each employs a distinct methodology to fuse sensor data and mitigate the effects of disturbances, thereby improving orientation estimation accuracy and robustness.

In this thesis, we build upon these established approaches by proposing a data-driven fusion filter. Instead of relying solely on analytical models of sensor error, our method leverages machine learning techniques to learn the patterns of noise and disturbances directly from data. By doing so, we aim to enhance orientation estimation performance beyond the capabilities of standard model-based fusion filters.

1.4 Orientation Estimation through Complementary Fusion

The Complementary Filter (CF) is one of the most basic forms of sensor fusion, where the orientation estimates from the angular rate gyroscopes are combined with the estimates generated from the accelerometer and, if available, the magnetometer. The basic complementary filtering approach [15], [17] is represented as:

$$R = (1 - \alpha)R_g + \alpha R_{am} \tag{1.1}$$

where R is the final rotation estimate, R_g is the rotation estimate from the gyroscopes, and R_{am} is the rotation estimate from the accelerometer and magnetometer. The rotation estimate R can be represented in multiple forms, such as Euler angles, quaternions, or rotation matrices.

1.4.1 Key Strengths of CF

- Reduced Calculation and computation, simple mathematical model.
- Faster convergence compared to Madgwick and EKF Filters[43].

1.4.2 Weaknesses of CF

- The value of the gain α is constant.
- Cannot efficiently differentiate between linear acceleration and gravitational acceleration.

1.5 Our Goal: Overcoming the Limitations of Complementary Filters Using a Data-Driven Approach

Complementary filters (CF) provide a simple and computationally efficient method for fusing MARG sensor data for orientation estimation. However, as discussed in section 1.4, CF is limited in handling sensor noise and varying dynamic conditions, particularly under high linear accelerations or magnetic disturbances. The objective of this work is to overcome these limitations by leveraging data-driven models to enhance the robustness and accuracy of CF.

The main contributions of this thesis are as follows:

- **Adaptive Alpha Fusion via Movement Prediction:** We propose a data-driven approach to dynamically toggle the CF parameter α. A decision tree-based predictor detects periods of movement, and based on its output, α is modulated: it is turned off for certain iterations and kept on for others, effectively translating the alpha variation into the frequency domain. This adaptive fusion strategy allows the filter to respond optimally to both static and dynamic conditions, enhancing accuracy and stability.
- **Noise Compensation via XGB Ensemble:** We introduce an Extreme Gradient Boosting (XGB) based ensemble model to predict and remove noise from gyroscope and normalized accelerometer-magnetometer (AccelMag) data. While tree-based models have been extensively used in other domains, their application for denoising MARG sensor data in orientation estimation is novel. The model efficiently learns complex noise patterns, providing a clean input for the complementary filter.

Together, these contributions place the complementary filter on a data-driven foundation, mitigating its traditional weaknesses and making it a more robust and competitive algorithm for real-world orientation estimation tasks.

1.6 Organization

After this introduction , we elaborate on some of the related works that have been done on this field in chapter 2. Then we describe our proposed methodology in chapter 3. Finally, we show some of the results we have achieved so far in chapter 5. The results which were acheived in chapter 5 are further elaborated and broken down in chapter 6. Lastly, chapter 8 discusses what we look forward to do in testing our proposed algorithm on real world physical scenarios instead of working on error metrics over a synthetic dataset.

Chapter 2

Related Works

From Virtual Reality domain and smartphone-based pedestrian navigation to autonomous aerial vehicles and human motion analysis, a wide range of applications across multiple domains rely on orientation estimation utilizing the IMUs (Inertial Measurement Units) incorporated in the devices. IMUs provide interpolated data collected from three sensors **(accelerometer, magnetometer and gyroscope)** and provide vital information regarding the angular rate, attitude/orientation or specific force for the body. The inherent drawbacks of MEMS (Micro-Electro-Mechanical-Systems) IMU sensors, such as noise, bias, and drift over time, make orientation estimation difficult and need the application of complex algorithms to fuse sensor data and generate precise and trustworthy orientation estimations. It directly influences subsequent computations like velocity and position estimation through *dead reckoning*. Dead reckoning is a phenomenon in which errors in orientation estimation can accumulate over time, leading to significant deviations in position tracking. Therefore it is all the more essential to enhance the accuracy and reliability for benchmarking the overall performance of the navigation and tracking system.

2.1 Approaches and algorithms

Several literature have addressed these problems in different ways i.e in *mathematical/deterministic* ways or through *data-driven ways using machine learning or deep learning*, or a *hybrid* of them. For example, Vertzberger and Klein note that smartphone IMUs incorporate high-amplitude noise and time-varying bias, and that achieving accurate estimation requires the involvement of all nine IMU signals (all three axes of the three inertial sensors) with time-varying weights. [44]. In their work they did not incorporate the magnetometer results. In practice, the filter must adjust to changing dynamics and varying noise, which fixed fusion weights often fail to address. Below are the approaches that have been used to address the challenges imposed by orientation estimation on the basis of a timeline:

2.1.1 Mathematical approaches

AHRS is an electronic system that provides information on an object's orientation, specifically pitch, roll, and yaw. A broad class of AHRS solutions use deterministic algorithms such as complementary filter[22], extended Kalman Filters (EKF)[31], unscented Kalman Filters (UKF)[47] and gradient descent [29] methods. These exploit known kinematic gyroscope readings and external reference vectors from accelerometer (gravity) and magnetometer (magnetics) in closed form. For instance, the QUEST algorithm[40][6] and Kalman filters solve Wahba's problem (erroneous rotation matrix calculation)[46] optimally, but assume stationary noise models. As stated in [44], Kalman filters are largely dependent on the tuning of the process and measurement covariance, and incur heavy computational cost due to calculations referring to inverse matrices. Conversely, complementary filters follow a linear interpolative approach and fuse gyro predictions with accelerometer/magnetometer corrections in the frequency domain. CFs (Madgwick [29], Mahony [30]) have intuitive fixed gains (alphas) and comparatively low computation, yet they suffer from slow convergence under rapid motions or shocks. Madgwick's filter [29] uses a gradient-descent step calculation based on accelerometer and magnetometer Jacobians, but requires manual tuning of the step gain (alpha). To address these problems regarding dynamic motions, variable-gain CFs have been proposed. Ding Duong Quoc et al. demonstrated the performance of this adaptive-gain CF [14] which dynamically changed the filter constant) and observed it to perform significantly better than a fixed-gain CF in roll/pitch accuracy under varying motion. Likewise, Shao et al. design a variable-gain complementary filtering method for aircraft angle-of-attack estimation, proving in simulation that continuously adaptive filters yield far more stable estimates than conventional CFs. It dynamically adjusts the alpha gain based on inertial network variance patterns, reducing the error by half compared to the fixed-gain designs in aerodynamic sensing applications. [37]. Similarly, Broughton et al. (2019) develop a robust complementary filter for UAVs that monitors the times when accelerometer readings are reliable. Based on the 'steadiness' model, it decides when to incorporate gravity measurements and constrains gyro biases using a Gaussian random-walk model. In Monte-Carlo tests of aggressive maneuvers, this filter has been able to track roll more accurately than standard CFs. [1]

Kalman filters make use of probabilistic models to recursively predict and update orientation estimates [31]. Sabatini's quaternion-based EKF incorporates sensor biases and mitigates motion and noise disturbances by adapting measurement noise covariances, which addresses the challenge of adaptability in human movement analysis [36]. The Unscented Kalman Filter avoids linearization by propagating sigma points, offering better performance when noises are Gaussian. [47]. However, these Kalman-based approaches require careful covariance tuning and are complex because of inverse matrices. In practice, researchers often trade theoretical optimality for robustness by using simpler linear filter (CF or gradient-descent) or by applying dual-filter schemes as demonstrated in [13]. In this work, a two-step EKF is performed on a 9 DOF device (accelerometer, magnetometer, gyroscope) which models disturbances explicitly and results in $<1°$ static error, stabilizing performance against magnetic disturbances. But despite high accuracy, EKF incur heavy computational performance overhead. But still, [44] surveys that the algorithm in [40] used for EKF [31] is probably the most accepted approach despite its sensitivy to noise assumptions.

Madgwick's gradient descent filter formulated orientation estimation as an optimization using Jacobians to minimize the error between predicted and measured sensor vectors [29]. It takes only 227 operations per update, which is significantly lighter than EKF implementations and enables high sampling rates in resource-constrained platforms. While the filter converges well under quasi-static conditions, it exhibits slow convergence under rapid dynamic motions because of the erroneous accelerometer bias from the fixed gain [49]. As opposed to the variable gain solutions previously discussed, an enhanced gradient descent algorithm was proposed by Wilson et. al (2019) which decoupled the magnetic variance from the calculation and improved the robustness for the convergence [49]. In order to mitigate magnetic disturbances and accelerate convergence, several enhancements have been proposed. Madgwick's extended complementary filter for full body MARG orientation (2020) dynamically adjusts gain parameters to decouple magnetometer influence, achieving up to 33% heading error reduction under fast-changing environments while maintaining computational efficiency [28].

Zhang et al. (2020) propose an **adaptive sparse-interpolation CF (ASICF)** for MEMS IMUs. They use a quaternion-based complementary filter for fusing gyroscope and accelerometer, but samples assesses the 'trustworthiness' of the accelerometer measurements (via its variation) from the successive samples, and if the data are too noisy or dynamic, it performs an interpolation step rather than using the raw sample [51]. Basically, the algorithm performs an *adaptive data-skip mechanism* and then fills the gaps produced from the dropped samples using interpolation. They evaluated ASICF on multiple datasets and report significantly lower attitude errors under large disturbances (20% smaller error than the Valenti CF [51]. Its performance depends directly on being able to detect outliers, and upon detection, it can handle sudden motions by effectively smoothing or skipping bad accel readings.

Guo et al. (2023) introduce a **variable gain CF** for aircraft angle-of-attack estimation using a distributed IMU network plus flush-air sensors [37]. They derive a flight-phase-dependent blending factor: the filter coefficient (analogous to alpha) is allowed to vary with the change rate of the angle-of-attack, placing more weight on inertial data at high dynamics. Simulation results for a high-speed UAV show this VGCF yields significantly smaller AoA error than constant gains (approximately 0.0058° vs. 0.0017° RMSE i.e > 2x improvement). However, it is quite domain-specific even though it exemplifies adaptive gain analogous to our approach.

In summary, classical filters are computationally efficient and theoretically grounded, yet they often require hand tuned gains or multiple stages to handle non-ideal IMU behavior and mitigate erroneous values.

2.1.2 Machine-Learning Enhanced Filters

The overhead for manual tuning is reduced by introducing data-driven machine learning approaches in orientation estimation. One approach is to make the filter gain *adaptive* using sensor data. Our approach is strongly analogous to the approach followed by Vertzberger and Klein (2022). They introduced a **hybrid adaptive complementary filter** that learns axis-specific accelerometer weights via neural networks [44]. They integrate gyro data classically (quaternion integration) and then apply a complementary update in each axis whose weight is predicted by a small neural network based on estimated linear accelerations. The method was evaluated on a smartphone IMU (60 two-minute sequences of walking activities in pocket, hand etc. with VI-SLAM Ground truth [44]) and compared against fixed gain filters like Madgwick, Mahony, AEKF etc. The learned filter yielded the lowest roll/pitch errors (10-37% better than classic filters on average). It adapted to dynamic motion via data-driven weight tuning, which outperformed its corresponding fixed gain filters. However, it requires offline labeled training, added complexity for neural net and most importantly, the yaw is not addressed (no magnetometer fusion). In a similar spirit, Maton et al. tune a CF gain indirectly; they embed filter gains in Mamdani fuzzy inference system (FIS) and use a genetic algorithm to optimize the membership functions so as to minimize velocity error against ground truth (obtained via MOtion CAPture devices). In an experimental robot dataset, GA-tuned gains reduce velocity and position errors [33]. While it does not specifically require explicit vision-based orientation, the tuning, again, is done offline and needs to be redone for new scenarios. Another line of work trains small neural models to train corrections; minimize noise. For example, Al-Sharman et al. (2019) train a neural model to enhance filter outputs by learning correction terms [38]. Brossard et al. introduce deep CNN not end-to-end, but to *denoise* gyroscope signals before integrating. Evaluated on EuRoC and TUM-VI datasets, the learned filter *outperforms state-of-the-art methods* and even beats top visual-inertial odometry algorithms in attitude accuracy [10]. But it requires large ground-truth datasets for training and the open-loop mode still drifts (converge slower) due to the complex design. [48] employs a neural network which was trained on diverse motion datasets for performing real-time attitude estimation from 6-DoF IMU data without requiring brute force tuning. It demonstrated generalization across diverse environments and sampling rates, but domain-specific optimization is required for peak performance [48]. Hybrid approaches, although sets a promising direction for data-based fusion algorithms, might impose challenges upon encountering new motions in the environment i.e it comes with the risk of overfitting.

2.1.3 Deep Learning Enhanced Filters

Beyond hybrids, fully deep end-to-end methods have been explored. These treat orientation estimation as a regression or sequence problem using powerful neural architectures. Convolutional networks (CNN) and recurrent networks (LSTM, GRU, transformers) have been trained to map IMU time-series to orientation quaternions. Golroudbari and Sabour proposed an end-to-end CNN + bi-directional LSTM that inputs raw 6-DOF (mag exclusive) IMU measurements and directly regresses quaternion angles [5]. It was evaluated on 7 different datasets (120+ hours of motion including EuRoC, TUM-VI) and reduced orientation root mean square errors by 20-50% vs. Madgwick and EKF baselines without per-dataset fine-tuning. Their network uses 1D convolution later for extracting local motion features over 0.5s windows, followed by bi-LSTM layers for modeling temporal context and finally aligning with the quaternion normalization layer. [5]. Tedaldi et al. (2014) fused gyroscope and accelerometer streams using multi-head attention. Demonstrating faster convergence (<0.2s) and 25% lower drift over 1 minute trajectories compared to Madgwick. [42] Brossard et al. (2020, NDAG 27) train dilated convolution networks to denoise gyroscope signals and perform open-loop dead reckoning. On EuRoC and TUM-VI benchmarks, their method surpasses top visual-inertial odometry systems in attitude estimation accuracy without any visual input. These attention maps reveal that during high dynamic or rapid motion settings, the model attends more to gyroscope results, paralleling the alpha toggling concept in [10]. In [45], they used PCA and trained a stacked denoising autoencoder on synthetic IMU noise to correct accelerometer and gyroscope biases online. Over 120 seconds, it resulted in a 40% reduction in Allan-variance-derived (statistical analysis for deriving nature of a drift) drift. This [38] employs a 1D U-net architecture to remove magnetic disturbances from MARG data, lowering heading error by 30% in indoor environments only with dynamic ferromagnetic interference as noise. These architectures can be refurbished to serve the purpose of noise prediction models. The common limitation of these deep-learning based approaches is the requirement for large ground-truth datasets and computational resources, as well as the "black-box" nature of these models. Nonetheless they demonstrate that fully learned orientation estimators can be highly accurate and adapt to unknown motion contexts, although these are quite heavy for low-cost sensors.

2.2 Gaps and Opportunities

Across these studies, most *adaptive* methods still rely on either **fixed fusion structures** with heuristic tuning or offline learning. For example, fixed-gain complementary filters (Mahoni [30], Madgwick [29] are cheap but cannot adjust to rapid dynamic motions, whereas hybrid approaches like [44] or [51] adapt to motion but require pre-training or brute force tuning. Pure learning methods in [10], [5] achieve excellent accuracy but need vast training data and lose interpretability. The genetic or fuzzy tuning of [33] shows that optimizing gain helps, but remains static once deployed. To the best of our knowledge based on the extensive reviews we have done, works that use machine learning to **predict sensor noise *online*** or to dynamically **toggle the complementary filter co-efficient in *real time*** do not exist. This constitutes a gap; a **a model based filter** whose parameters (measurement noise or blending factor α) adapt instantaneously to the current operating conditions based on learned patterns. Our XGBoost approach is able to fill this gap by estimating noise levels (and thus adjust α) on the fly, combining the strengths of data-driven adaptivity with the structure of a complementary filter. This addresses dynamic disturbances in a principled way that is not covered by the existing literature in [44] and [10].

2.3 Cohesion with Approach

We are following two main approaches for conducting the orientation estimation. The first one involves an XGBoost trained Toggle Engine that sets α-gain=0 when it detects motion letting the gyro integration alone drive the rapid changes, and α-gain

> 0 during rest to fuse accelerometer and magnetometer values for drift correction. In coherence with this, Meyer et al. [1] detect dynamic vs. steady states via gyro-bias and mag-rate thresholds, improving heading error by 33% under fast maneuvers. Likewise, Shao et al. [37] dynamically adjusted α based on inertial network variance, halving angle-of-attack error compared to its fixed gain substitutes. Most importantly, the literature that corresponds closest to our approach, [44] by Vertzberger and Klein embed a neural net to predict axis-specific gains, outperforming static α-filters across human motions from pedestrian data. [42] shows multi-head attention naturally attends to gyro during high dynamics, which is analogous to setting α as 0 when motion is detected by our Toggle Engine. Similar approaches have been followed in [19], [49], [5] but all of them one way or other fine-tuned the parameters required offline as a part of pre-processing, whereas we are following a approach that predicts the motion and dynamically adjusts the α gain online. Again, the hybrid or ML/DL-based models rely on multilayer perceptron, but we use XGBoost. The tree-based model is far lighter and feasible for embedding in a micro-controller, whereas the neural networks require more memory and FLOPs. For our second approach, we are essentially denoising using XGBoost. We are trying to predict the noise (the quaternion error between the complementary filter output and the ground truth) and at runtime, we subtract this derived noise from the CF-derived vectors before the next fusion step. In coherence with this, [51] predicts accelerometer noise via XGBoost, cutting Allan variance by 40% vs. that in ARMA (Auto-Regressive Moving Average). [28] suffered from slow convergence as stated in [29], but the fixed β (read α), cannot adapt to noise fluctuations, Our XGBoost noise regressor can suggest β adjustments via equivalent covariance tuning. Again, survey papers related to [31] state that static covariance assumptions limit filter robustness; which motivates our use of XGBoost for predicting time-variance noise statistics for EKF covariance updates. In [51], they train a 1-D U-net to remove magnetometer disturbances in indoor MARG, which is essentially equivalent to an XGBoost regressor per axis for disturbance correction, feeding EKF or Complementary signals with cleaner signals. So rather than static or CNN-based denoising, our XGBoost regressor directly target the CF quaternion-error preserving low compute and interpretability while achieving deep-net level noise reduction. Neural networks (CNN, LSTM, BiLSTM) being used in [5] and others while providing high accuracy, are black-boxes with millions of parameters, whereas we can ensure interpretability by incorporating a simple if-else based tree and the outputs can be linked to the inputs, which is why it easier for our model to run on low power embedded models in real time.

Chapter 3

Proposed Methodology

In this chapter, we propose a data-driven methodology designed to overcome the limitations of the Complementary Filter (CF). Thus the following section will correspond to chronological steps executed to improve the α prediction and noise elimination.

3.1 Research Design

We will go through six steps, where we introduce our mathematical background for the filter design and model training. The steps are listed below:

- Modified Complementary Filter Design based on the works of [44].
- Training Dataset
- Training the ML model for movement prediction
- Training the ML model for noise prediction
- Evaluation of the proposed filter on the BROAD [26] dataset.
- Evaluation of the proposed filter on the Mechanical Turntable platform as described in section 4.1.

These steps are elaborated in later sections.

3.2 Choice of the Machine Learning Model

For our research, it is critical that the models run in every iteration to both denoise sensor readings and predict movement. This requirement necessitates models with very low inference time, making tree-based approaches particularly well-suited for our needs.

Additionally, it is essential that the models maintain a favorable bias-variance tradeoff and demonstrate robustness to outliers. While tree-based models can be prone to overfitting [9], employing a robust learning strategy is necessary to mitigate this risk. In this work, we adopt the RANSAC (Random Sample Consensus) algorithm [16] to enhance the robustness of the XGBoost trees.

A detailed discussion of the reasoning behind the choice of these models is provided later in this section.

3.2.1 Choice of Extreme Gradient Boosting Regression and Classification Models

XGBoost (Extreme Gradient Boosting) has become a preferred choice for both regression and classification tasks due to its robust performance, scalability, and advanced regularization techniques. The key reasons include:

- **High Predictive Accuracy:** XGBoost consistently achieves impressive performance on structured/tabular datasets in both academic benchmarks and real-world competitions due to its ability to minimize bias and variance simultaneously [12]. Such accuracy is required for our experimentation for accurate movement prediction.
- **Regularization Capabilities:** Unlike traditional gradient boosting, XGBoost includes L1 and L2 regularization terms, which help prevent overfitting—a common issue in high-dimensional data [12].
- **Handling of Missing Values:** XGBoost can handle missing data internally during training without requiring preprocessing imputation, improving ease of use and robustness [12].
- **Efficient Computation:** With its use of histogram-based algorithms and parallel processing, XGBoost is highly efficient and scalable to large datasets [24]. BROAD dataset with almost 2,400,000 records combined, needs a highly efficient and paralellizable model training process.
- **Flexibility:** It supports a variety of objective functions and custom loss functions, making it suitable for a wide range of regression and classification problems [12].

Figure 3.1: Regression Model Training Pipeline with RANSAC

3.2.2 Choice of Random Sample Consensus (RANSAC) for Regression

Random Sample Consensus (RANSAC) is an iterative algorithm designed to estimate the parameters of a mathematical model from a dataset that may contain outliers. RANSAC operates by selecting random subsets of the data to fit a model and then determining the number of inliers—data points that fit the model within a predefined tolerance. The model with the highest number of inliers is considered the best fit. The Random Sample Consensus (RANSAC) algorithm is effective in producing stable model estimates by focusing on a consensus subset of the data [16]. In terms of variance in the BROAD dataset, this approach reduces the variability of parameter estimates, leading to more reliable and robust models that better capture the underlying data distribution.

XGBoost can be sensitive to outliers in the training data, which may lead to overfitting and reduced generalization performance. Integrating RANSAC with an XGBoost regressor combines the strengths of both methodologies to enhance model robustness and predictive performance. The regression training pipeline with RANSAC is visualized in **??**:

- **Robustness to Outliers**: RANSAC's ability to identify and exclude outliers ensures that the XGBoost model is trained on cleaner data, leading to more reliable predictions.
- **Enhanced Generalization**: By focusing on inliers during the training process, the combined model is less likely to overfit to noise and anomalies, improving its performance on unseen data.
- **Complementary Strengths**: While XGBoost excels at capturing complex, non-linear relationships in data, RANSAC provides a mechanism to mitigate the influence of outliers, resulting in a more robust and accurate model.

This hybrid approach has been explored in [27], demonstrating its effectiveness in improving model performance in the presence of outliers.

3.3 Modified Complementary Filter Design

The complementary filter algorithm we incorporated in our thesis is described in this section. The CF filter featured here incorporates a gain parameter α which would

Figure 3.2: Filter Diagram[1] for our proposed Data Driven Toggling Gain CF.

be generated by a *Toggling Engine* mentioned in section 3.5. The algorithm also supports noise removal if a trained model is provided[10].The algorithm represents orientation using vector-based measurements from the accelerometer and magnetometer. The raw accelerometer vectors are fused with the gyroscope-based rotation from the previous estimate, following the approach in [44]. In contrast to this prior work, we extend the method by incorporating magnetometer readings, refining the prediction of the fusion parameter α, and introducing a dedicated noise removal step.The full algorithm description of our proposed filter can be found in algorithm-1. Some information about the algorithm:

- The algorithm uses the a_{pred} and m_{pred} (predicted gravity and north vectors respectively) to describe the orientation. This representation can be interchanged to quaternion or a rotation matrix[8][11].
- The sensor readings are formatted as $(3, 1)$ column vectors:
  - Gravitational Vector from the accelerometer: $a_i = (a_{ix}, a_{iy}, a_{iz})$
  - Magnetic North Vector from the magnetometer: $m_i = (m_{ix}, m_{iy}, m_{iz})$
  - Gyro angular rates on 3 axis: $g_i = (g_{ix}, g_{iy}, g_{iz})$
- The rotation matrix generated in the iteration i from the angular rates [50] is:

$$R_g = e^{(\Omega_\times(g_i)\delta t)} \tag{3.1}$$

  Where $\Omega_\times$ is the skew-symmetric operator and δt is the sampling rate.

The algorithm is represented through the flow diagram in **??**

3.4 Dataset Description

This section describes the dataset we are currently utilizing to train our models. The dataset we are using is the Berlin Robust Orientation Estimation Assessment Dataset (BROAD)[26].

The BROAD dataset contains 39 recordings or trial. Each of the trial contains a time series of MARG sensor data readings along with the boolean parameter `movement` and the position(`opt_pos`) and the orientation(`opt_quat`) information of each timestamp gathered from the Optitrack OMC system at 120Hz.

The MARG sensor readings come from a custom 3D printed device on which an Inertial Measurement Unit (IMU) is mounted. The IMU was a commercially available nine-axis inertial sensor (myon aktos-t, myon AG) recording 9 sensor reading at a sampling rate of 286Hz. The hardware device is shown in Figure 3.3(b). The trials in the dataset are generated through the motion of the 3D printed device through different pre-defined motion parameters which is elaborated in the paper for BROAD dataset. A brief categorization of the types of recording is shown in Table 3.1. This categorization is based on the type of physical motion the 3D printed hardware goes through while undergoing a pre-defined motion trajectory while recording the trial.

Table 3.1: Categorization of BROAD Dataset Trials

Motion Type	**Speed**	**Trial/recording Indexes**
Undisturbed		
Rotation	Slow	01, 02, 03, 04, 05
Rotation	Fast	06, 07, 08, 09
Translation	Slow	10, 11, 12, 13, 14
Translation	Fast	15, 16, 17, 18
Combined	Slow	19, 20
Combined	Fast	21, 22, 23
Disturbed (Medium Speed)		
Tapping	–	24, 25
Vibrating Smartphone	–	26, 27
Stationary Magnet	–	28, 29, 30, 31
Attached Magnet (1–5)	–	32, 33, 34, 35, 36
Office Environment	–	37, 38
Mixed (Disturbed and Undisturbed)	–	39

[1]The colors in the diagram have no significance.

Algorithm 1: Modified CF algorithm

Input: MARG sensor readings = $\{(g_1, a_1, m_1), (g_2, a_2, m_2), ...\}$, Toggling Engine, Noise Model, Alpha: α, Duty Cycle Params: d_l and d_h

Output: Predicted gravity and north vectors a_{pred} and m_{pred}

Calibration sample count: $N = 500$

Gyro bias: $g_{bias} = \sum_{i=0}^{N} g_i$

Initial Gravity Vector Prediction: $a_{pred} = \sum_{i=0}^{N} a_i$

Initial North Vector Prediction: $m_{pred} = \sum_{i=0}^{N} m_i$

foreach (g_i, a_i, m_i) *in MARG sensor Readings,* $i > N$

do

Remove gyro bias and normalize accel-mag readings:

$$g_i = g_i - g_{bias}$$
$$a_i = normalize(a_i)$$
$$m_i = normalize(m_i)$$

Form R_g from g_i.

Form the predictions from the gyro sensor:

$$a_{gyr} = R_g \cdot a_{pred}$$
$$m_{gyr} = R_g \cdot m_{pred}$$

Predict noise from noise model:

$$(a_{noise}, m_{noise}) = NoiseModel(g_i, a_i, m_i)$$

Predict alpha from the *Toggling Engine*:

$$\alpha = TogglingEngine(i, g_i, a_i, m_i)$$

Compute the dynamic scaling values:

$$e_{ga} = ||\mathbf{a} - \mathbf{a}_g||, \qquad e_{gm} = ||\mathbf{m} - \mathbf{m}_g||, \qquad e_{na} = ||\mathbf{a}_{noise}||, \qquad e_{nm} = ||\mathbf{m}_{noise}||$$
$$\alpha_{ga} = \frac{\alpha \cdot e_{ga}}{e_{na} + e_{ga}}, \qquad \alpha_{gm} = \frac{\alpha \cdot e_{gm}}{e_{nm} + e_{gm}}, \qquad \alpha_{na} = \frac{e_{na}}{e_{na} + e_{ga}}, \qquad \alpha_{nm} = \frac{e_{nm}}{e_{nm} + e_{gm}}$$

Update the predictions:

$$a_{pred} = a_{gyr} - \alpha_{ga}(a_i - \alpha_{na} a_{noise} - a_{gyr})$$
$$m_{pred} = m_{gyr} - \alpha_{gm}(m_i - \alpha_{nm} m_{noise} - m_{gyr})$$

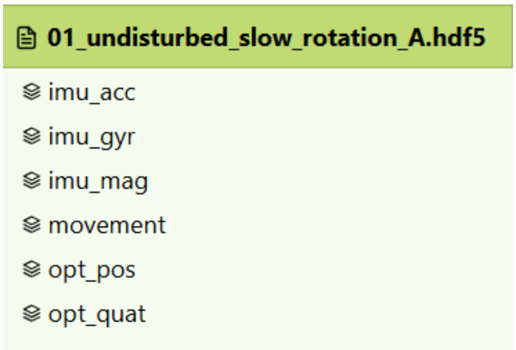
Normalize the predictions:

$$a_{pred} = normalize(a_{gyr})$$
$$m_{pred} = normalize(m_{gyr})$$

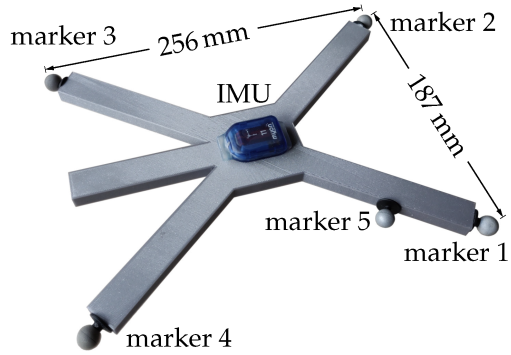
Append a_{pred} and m_{pred} to $orientationEstimates$

end

return $orientationEstimates$



(a) Columns in the BROAD dataset



(b) 3D printed hardware device with strapped on IMU.Optical markers placed on four corners for tracking

3.5 Gain(α) Estimation through Movement Prediction

Here we describe the working principle of the *Toggling Engine*. The α parameter is initially set to be a constant value. The toggling engine is responsible for toggling the alpha in presence of movement. Detailed steps:

- Train an XGBoost Tree Classifier[12] model on the input features X where X contains the raw sensor readings of the MARG sensors and output class label y where y contains the boolean variable movement.
- Use the trained Model in the *Toggling Engine* according to the algorithm 2.

Algorithm 2: Toggling Engine

Input: MARG Sensor Readings: (g_i, a_i, m_i), Iteration Index i

Data: Alpha: α=0.01, Duty Cycle High: d_h, Duty Cycle Low: d_l,XGB Model $\mathcal{M}$

Output: Estimated α

Function `TogglingEngine(`i,g_i,a_i,m_i`):`
    Compute total period: $T \leftarrow d_h + d_l$
    Predict movement: $\hat{\delta} \leftarrow \mathcal{M}(g_i, a_i, m_i)$
    if $\hat{\delta} == 0$ **then**
        return α
    if $(i \bmod T) < d_h$ **then**
        $\alpha \leftarrow \alpha$
    else
        $\alpha \leftarrow 0$
    end
    return α

3.6 Noise Removal

Here we describe the training procedure of the model that removes noise. The ground truth noise values are established from the difference of actual sensor measurements and the ground truth gravitational and magnetic north vectors. The ground truth quaternion q_{gt} generated from the quaternion components in `opt_quat` in the dataset is used to rotate the a_{avg} and m_{avg} vectors to remove the noises. The a_{avg} and m_{avg} vectors are calculated by averaging initial sensor readings for $N = 500$ iterations. The value of N could be varied.

- Ground truth gravity vector is calculated using the ground truth q_{gt} obtained for each record in a recording dataset `opt_quat`: More information on this rotation formula in subsection 9.1.2

$$a_{gt} = q_{gt}^{-1} \cdot a_{avg} \cdot q_{gt} \tag{3.2}$$

- Ground truth magnetometer north vector is calculated using the ground truth q_{gt} obtained for each record in a recording dataset `opt_quat`:

$$m_{gt} = q_{gt}^{-1} \cdot m_{avg} \cdot q_{gt} \tag{3.3}$$

- An XGBoost Regression model[12] is to be trained with input features X being the raw MARG sensor readings and the output feature y being the six components of the a_{gt} and m_{gt} vectors. The training dataset contains may contain all the MARG sensor readings from all the recordings, each sensor reading corresponds to a ground truth quaternion q_{gt} represented as `opt_quat` in the datasets. Each of the q_{gt} is used to generate a_{gt} and m_{gt} vectors used for the regression model training. The loss function used is the generic *Mean Squared Error Loss function* [7]. The trained model is to be used to do noise prediction in algorithm 1 where the noise is subtracted from the sensor readings.

- The noise prediction is done based on recordings in the BROAD dataset[26].Thus there exists a possibility of the noise removal algorithm being biased towards the sensor setup shown in Figure 3.3(b) which has been used to record the readings in the BROAD dataset.

3.7 Evaluation on the Dataset

The proposed filter algorithm was evaluated using the BROAD dataset [26]. Two configurations of the filter were tested: one with the machine learning (ML)-based denoising module enabled, and another with the ML module disabled. For comparison, the standard implementation of the Madgwick filter [29] was also included in the evaluation.

The BROAD dataset provides ground-truth orientation in the form of quaternions. From these ground-truth quaternions, the error between the estimated orientation and the true orientation was computed using the quaternion absolute distance metric. The error is expressed in radians and is formally defined as:

$$e_\theta = d_Q\left(\hat{q}, q_{gt}\right) \tag{3.4}$$

where $\hat{q}$ denotes the estimated quaternion, q_{gt} denotes the ground-truth quaternion, and $d_Q(\cdot,\cdot)$ represents the quaternion absolute distance metric. The detailed formula can be found in section 9.2. This metric provides a scalar angular error that quantifies the rotational difference between the estimated and ground-truth orientations.

3.8 Evaluation on the Mechanical Turntable

The evaluation on the mechanical turntable follows the same metrics as the dataset-based evaluation described in Section 3.7. The turntable, whose design is detailed in section 4.1, enables controlled single-axis rotational motion. The ground-truth orientation is obtained from the AS5600 angle encoder, which provides angular measurements that are converted into quaternions for comparison.

The evaluation procedure can be summarized as follows:

- **Ground Truth Acquisition:** The AS5600 absolute magnetic encoder is used to record the turntable angle. These measurements are transformed into ground-truth quaternions.

- **Parallel Filter Execution:** Multiple orientation filters were executed simultaneously, including AQUA [43], EKF [34], Madgwick [29], and the proposed filter. Each filter estimated orientation from the MPU-9250 MARG sensor data acquired through a Raspberry Pi.

- **Error Metric:** For each filter, the quaternion absolute distance (see Appendix 9.2) was computed between the estimated quaternion $\hat{q}$ and the ground-truth quaternion q_{gt}. This scalar error, expressed in radians, quantifies the rotational difference.

- **Result Visualization:** The error traces over time were plotted and are reported in the Results section (Section 5.4), enabling direct comparison of filter performance.

Chapter 4

Experimental Setup

4.1 Mechanical Platform (Turntable)

4.1.1 Electronics and Sensors

The inertial sensing subsystem employed in this work is based on the InvenSense/TDK MPU-9250, which integrates a tri-axial accelerometer, gyroscope, and the AK8963 tri-axial magnetometer. Each sensing axis utilizes a 16-bit ADC, providing sufficient dynamic range for motion tracking. In addition, an AS5600 magnetic angle encoder was used to obtain ground-truth angular position measurements. Data acquisition and processing were handled by a Raspberry Pi 5 (8 GB) running the default Raspberry Pi OS. Communication with both the MPU-9250 and the AS5600 encoder was established over the I^2C bus at a logic level of 3.3 V. Power for the entire setup was supplied by a 20,000 mAh USB power bank. The integration of these sensing and processing components into the turntable setup is detailed in Subsection 4.1.2.

- **IMU:** InvenSense/TDK MPU-9250, tri-axial accelerometer and gyroscope with integrated AK8963 magnetometer, 16-bit ADCs.
- **Gyroscope ranges:** $\pm$250, $\pm$500, $\pm$1000, and $\pm$2000 dps (default range used).
- **Accelerometer ranges:** $\pm$2, $\pm$4, $\pm$8, and $\pm$16 g (default range used).
- **Sample-rate and DLPF settings:** Default sample rate and default digital low-pass filter configuration were used.
- **Encoder:** AS5600 magnetic angle encoder for ground-truth angular position.
- **Processor/DAQ:** Raspberry Pi 5 (8 GB) running the default Raspberry Pi OS.

- **Communication:** I^2C bus, 3.3 V supply voltage and logic level.
- **Power:** 20,000 mAh USB power bank.

4.1.2 Turntable Setup

The experimental turntable was designed in Fusion360 as shown in Figure 4.1(b) and constructed using 3D-printed parts and integrated sensors to provide both actuation and ground-truth angle measurement. The constructed device is shown in Figure 4.1(a) and Figure 4.1(c). The system consists of a circular rotating disk mounted on a bearing, a box-shaped support structure, and a base platform with a magnet for the AS5600 magnetic encoder. The disk carries the electronics, including a Raspberry Pi 5, an MPU9250 MARG sensor, and a 20,000 mAh power bank, which provides standalone operation. The detailed specifications of the turntable are summarized below.

- **Turntable geometry:** Diameter [28 cm], 3D printed PLA/ABS material, mass [1.5 kg] approximately with everything mounted on the platform.
- **Actuation:** There is no electrical actuation built into the turntable, rather movement is executed by manually rotating the turntable.
- **Mechanical construction:**
  - Upper circular disk connected to the inner ring of the 6010 bearing.
  - Box-shaped 3D printed housing supporting the bearing.
  - Platform with embedded magnet for AS5600 encoder.
- **Ground-truth sensing:** AS5600 absolute magnetic encoder with the following characteristics:
  - Resolution: 12-bit (4096 positions per revolution, $\approx 0.087°$ per step).
  - Angular accuracy: typically $\pm 0.4°$ as shown in [2], depending on magnet alignment and air gap.
  - Maximum angular deviation: up to $\pm 1.4°$ worst-case under misalignment or offset conditions.
  - Latency: on the order of 1–2 ms (limited by I^2C polling rate or PWM readout frequency).
  - Output interfaces: I^2C, PWM, or analog voltage.

- **On-disk payload:** 20,000 mAh powerbank, Raspberry Pi 5 (8 GB), MPU9250 MARG sensor, AS5600 encoder.

4.1.3 Motion Protocols

The turntable experiments were conducted under three categories of motion: static, quasi-static, and dynamic[20]. The coordinate frames tested were determined by the feasible orientations of the setup: rotation was applied around the z-axis and y-axis with the positive direction pointing upward, and around the x-axis with the positive direction pointing downward. Rotation around the x-axis upward was not performed due to mechanical constraints imposed by wiring. The three types of motions are described below:

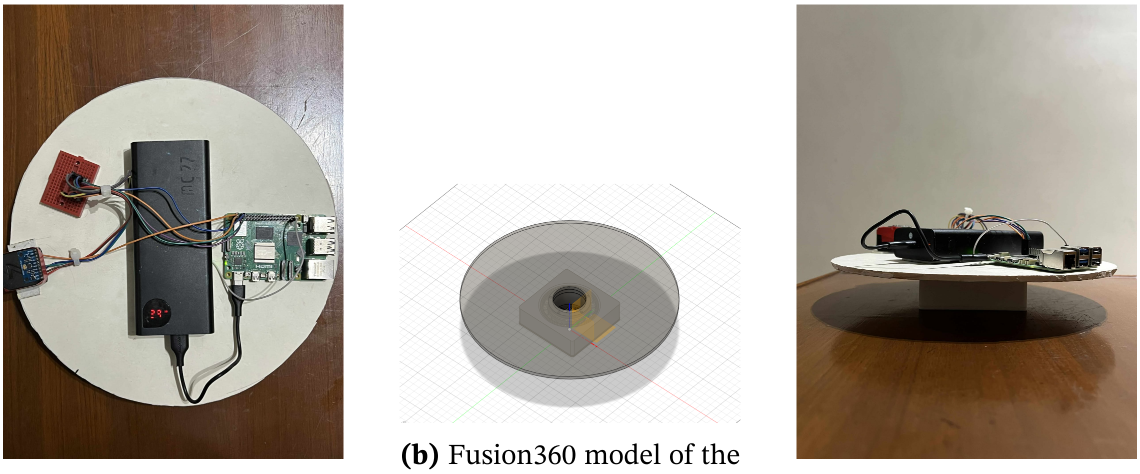
- **Static:** The system experiences no movement. This condition is used to establish baseline sensor stability and drift characteristics.
- **Quasi-static:** Slow and controlled movements are applied, typically at speeds low enough that inertial effects are negligible. This allows the evaluation of sensor accuracy under gradual angular changes. The turntable supports motion along only one axis at a time, so results may vary depending on the axis under test.
- **Dynamic:** Continuous and rapid motions are applied, where inertial effects are significant. This tests the system response under sustained rotation and higher angular velocities. Since the turntable provides single-axis rotation only, the evaluation is performed one axis at a time, and outcomes may differ depending on which axis is being tested.

4.1.4 Software Setup and Result Generation

The software infrastructure for the experimental turntable was implemented on a Raspberry Pi 5 (8 GB). The system executes four independent Python scripts corresponding to different orientation estimation algorithms: the proposed filter, the AQUA filter, the Extended Kalman Filter (EKF), and the Madgwick filter. These scripts are coordinated by a parent `run` script, which launches each of the four filter scripts in separate threads to allow concurrent execution.

Each filter script reads sensor measurements from the MPU-9250 MARG sensor via the I^2C protocol, capturing tri-axial accelerometer, gyroscope, and magnetometer data. Read concurrency exists while reading the data from the sensor, but as the read-time is short, the filters can ultimately run in the required refresh rate parallely. In parallel, a dedicated script reads angle measurements from the AS5600 magnetic encoder and converts the encoder readings into a ground-truth quaternion representation of the turntable orientation.

During runtime, the estimated quaternions from the four filters, along with the ground-truth quaternion, are continuously recorded into CSV files. Post-processing involves computing the quaternion absolute distance metric as in section 9.2, between the ground-truth quaternion and each filter's estimate. These distance metrics are then plotted to visualize and compare the performance of the different orientation filters under various motion conditions. The summarized version of the results are shown in section 5.4. The graphs of the results can be found in the appendix.



(a) Turntable(Top View)

(b) Fusion360 model of the turntable

(c) Turntable(Side View)

Figure 4.1: Overview of the turntable setup with key components highlighted.

Table 4.1: Summary of motion protocols applied to the turntable

Motion Type	Description
Static	No movement; turntable held stationary.
Quasi-static	Rotation from 0° to 180° clockwise, then 0° to 180° counterclockwise; break; repeated once (total of two cycles).
Dynamic	Continuous full rotations: 3 turns, break, 4 turns, break, 5 turns.

4.2 Dataset

The dataset was already presented in section 3.4. Based on this the experimental pipeline was generated. The experiment is carried out in 3 parts, in the initial two parts the models for the movement and noise prediction are trained on the BROAD[26] dataset. Then the trained model is used to implement the Modified CF as of algorithm 1 and the results are reported as absolute distance between the Ground truth and the predicted rotation quaternions[23].

4.3 Training

In this section we provide the methodologies followed to train the models to predict noise and movements.

4.3.1 Training to Predict Movement

This subsection focuses on the training of the model to predict the movement states from the reported sensor readings. Detailed steps:

- There are 39 (indexed from 0 to 38) recordings in the BROAD dataset. For our experiments, we selected the following subset of recordings, the `concat_list` (the set contains the indices):

$$\{0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 15, 16, 18, 19, 20, 21, \\ 23, 24, 25, 26, 27, 28, 29, 31, 32, 33, 34, 35, 36, 37, 38\}.$$

Table 4.2: Dataset Generation to Predict Movement

X	**y**
All records/samples of MARG sensor readings from the concatenated dataset	Corresponding movement column of the concatenated dataset

- For each recording in `concat_list`, we follow the procedure described in section 3.5 to extract the MARG sensor readings along with the corresponding movement labels. Each processed recording generates a new CSV dataset whose columns contain the 9 raw MARG sensor readings and the associated movement label. Repeating this process for all recordings in `concat_list` produces 34 new CSV datasets.

- All 34 datasets are then concatenated to form one large dataset. From this concatenated dataset, the input features `X` and output labels `y` are generated according to Table 4.2.

- The dataset formed by the concatenation of recordings in `concat_list` is divided into training and testing sets with an 80–20 split ratio. The remaining recordings (those not included in `concat_list`) are reserved and used for validation. The choices of indices have been made in such a way so that a variety of motions can be included.

- The choice of model for this task was the XGBoost Classifier, as introduced in subsection 3.2.1. This model was chosen for its efficiency and scalability in handling large datasets and its strong performance on tabular sensor data.

- To further enhance the performance of the classifier, an `Oputna Study` library[4] was used for hyperparameter optimization. The study objective was to maximize the `accuracy_score`[25] between the predicted movement labels `y_pred` and the ground-truth labels `y_test` on the test set. The hyperparameter search space for the XGBoost Classifier is shown in Table 4.3:

Table 4.3: Choice of Ranges of Hyperparams for *Optuna Study(For Movement)*

Hyperparameter	**Description**	**Range**
`colsample_bytree`	Fraction of features to sample per tree.	[0.0, 1.0]
`learning_rate`	Step size shrinkage to prevent overfitting.	[0.0, 1.0]
`max_depth`	Maximum depth of a tree.	[2, 15]
`n_estimators`	Number of boosting rounds.	[1, 15]
`reg_alpha`	L1 regularization term on weights.	[0, 10]
`reg_lambda`	L2 regularization term on weights.	[0, 10]
`subsample`	Fraction of samples to use per tree.	[0.6, 1.0]

- The Optuna study was executed for `N=100` iterations, during which the model was trained within the study objective using the training subset. The study selected the best combination of hyperparameters that maximized the classification accuracy.

- Finally, the XGBoost Classifier was retrained using the best hyperparameters returned from the study object, forming the final trained model for predicting movement states from raw MARG sensor readings.

4.3.2 Training to Predict Noise

This subsection focuses on the training of model to predict the noises among the reported sensor readings. Detailed steps:

- There are 39 (indexed from 0 to 38) recordings in the BROAD dataset. For our experiments, we selected the following subset of recordings, the `concat_list` (the set contains the indices):

$$\{0, 1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 15, 16, 18, 19, 20, 21, \\ 23, 24, 25, 26, 27, 28, 29, 31, 32, 33, 34, 35, 36, 37, 38\}.$$

- For each recording in `concat_list` we follow the procedure mentioned in section 3.6. We average the first N entries then generate `a_noise` according to Equation 3.2 and `m_noise` according to Equation 3.3 for each record `i`. Then generate a new csv dataset for each recording whose columns contain 9 MARG sensor raw values and 6 noise values (3 component from `a_noise` and 3 component from `m_noise`). Doing it for each recording in `concat_list` generates 34 new csv datasets. We concatenate all of them to form a large dataset.
- Generate `X` and `y` from the large dataset according to Table 4.4

Table 4.4: Dataset Generation to Predict Noise

X	**y**
All records/samples of MARG sensor readings from the concatenated dataset	Corresponding 6 values of a_noise and m_noise

- Choice of model was **XGB regressor** but coupled with the **Random Sample Consensus(RANSAC)**[16] algorithm. The RANSAC algorithm made training the base XGB regressor more robust to outliers and it improved the bias variance tradeoff.
- Split the dataset formed by the concatenated dataset (the remaining recording files would be used for validation) into train and test sets with an 80–20 split ratio. Generate an optuna study[4] where the model is trained within the study objective on the train set and the objective works to maximize the R^2 score on the test set. The choice of hyperparameters is the same as in Table 4.5. The study runs for 200 iterations.

Table 4.5: Hyperparameter Search Space for XGBoost Regressor with RANSAC

Hyperparameter	**Type**	**Range**	**Description**
`colsample_bytree`	Float	[0.0, 1.0]	Subsample ratio of columns for each tree
`learning_rate`	Float	[0.0, 1.0]	Step size shrinkage used in update to prevent overfitting
`max_depth`	Integer	[2, 15]	Maximum depth of a tree
`n_estimators`	Integer	[1, 25]	Number of boosting rounds
`reg_alpha`	Float	[0.0, 10.0]	L1 regularization term on weights
`reg_lambda`	Float	[0.0, 10.0]	L2 regularization term on weights
`subsample`	Float	[0.6, 1.0]	Subsample ratio of the training instances
`min_samples`	Float	[0.1, 0.9]	Minimum number of samples for a model to be considered valid in RANSAC
`residual_threshold`	Float	[1e-3, 10.0]	Maximum residual for a data point to be classified as an inlier in RANSAC
`max_trials`	Integer	[100, 500]	Maximum number of iterations for RANSAC algorithm

- The final model is trained with the best hyper parameter returned from the `study` object.

4.3.3 Implementation and Testing of the Modified Complementary Filter

The complementary filter with all its modifications proposed in algorithm 1 were implemented using `python`. Separate classes were constructed for the *Toggling Engine* and *Complementary Filter* implementation. Code of the actual implementaion could be found here[1].

After implementation, the filter was fed with sensor readings from `cv_set` containing $\{8, 14, 17, 22, 30\}$ where each value in the set correspond to the index of the recording file not in the `train_set`.The `cv_set` is being used for validation. Each of the recording file fed through the filter gave us a quaternion estimation for each iteration, i.e. the

[1] `https://github.com/azfarus/NDAG_MARG_THESIS_v2`

file `01_undisturbed_slow_rotation_A` has approximately 56,000 sensor readings. Each sensor reading will correspond to a quaternion estimation(q_{esti}) and a ground truth(q_{gti}) quaternion. The error metric per iteration stands to be:

$$e_i = Quat_{abs\_dist}(q_{gti}, q_{esti}) \tag{4.1}$$

More information about the metric is provided in section 9.2. We compare the mean of all e_i found over a particular recording for various standard filters such as the Madgwick and the Complementary filter to our proposed filter.

Choice of the Optimal Hyperparameters

The selection of hyperparameters for the proposed filter was carried out through a combination of empirical analysis and grid search. The final choices were informed by sensor characteristics (e.g., gyroscope drift) and systematic parameter tuning. The procedure is summarized below:

- **Alpha (α):** This parameter was selected based on the observed gyroscope drift. A lower gyroscope drift allows for a smaller α, thereby placing greater trust in the gyroscope signal. In practice, α was tuned in the range [0.0001, 0.001], with the optimal value determined as $\alpha = 0.001$ for stable performance.
- **Cycles while alpha included (d_h):** Determined through grid search, the high-pass filter threshold was optimized to $d_h = 120$. This choice provided the best trade-off between responsiveness to dynamic motion and robustness against noise.
- **Cycles while alpha excluded (d_l):** Similarly, the grid search yielded an optimal value of $d_l = 60$ for the low-pass filter threshold, ensuring smooth filtering of quasi-static orientation components.

The results of the grid search procedure used to identify the optimal d_h and d_l parameters are illustrated in Figure 4.2, which shows the error (in Degrees, metric was defined in section 9.2) surface as a function of the threshold parameters.

Dataset Dependence

The hyperparameters for the filter selected in section 4.3.3 were optimized using the cross-validation set described in subsection 4.3.3. While this procedure ensures that the chosen parameters achieve good performance on the available data, it also introduces the risk of dataset dependence. In other words, the filter may be overly tuned

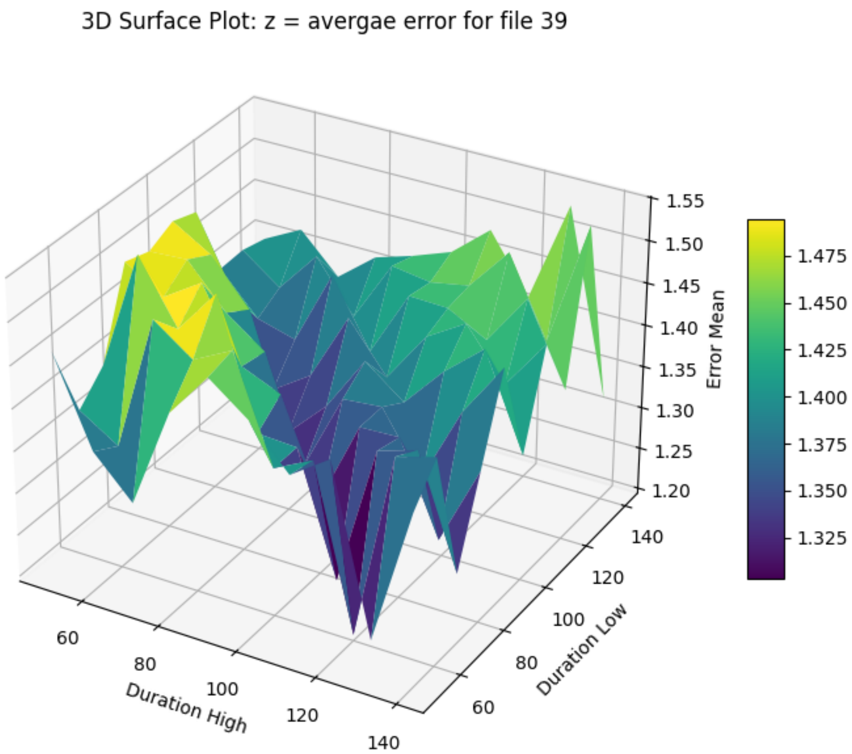


Figure 4.2: Grid search results for hyperparameter tuning of d_h and d_l. The optimal configuration corresponds to $d_h = 120$ and $d_l = 60$.

to the statistical properties, noise characteristics, or motion patterns specific to the training dataset.

Such dependence can limit the generalizability of the algorithm when applied to other datasets or real-world scenarios that exhibit different sensor noise profiles, motion dynamics, or environmental disturbances. Moreover, hyperparameters derived from one dataset may not translate directly to physical devices equipped with different sensor hardware. For example, variations in sensor quality, measurement ranges, bias stability, and filtering characteristics between IMUs can cause the optimized parameters to perform suboptimally when deployed on alternative devices.

Chapter 5

Results

5.1 Results of Noise Prediction

The XGB regression model run according to subsection 4.3.2 showed positive result in being able to predict the amount of noise in test set. The test set contained the recordings with indexes `cv_set=[8, 14, 17, 22, 30]`. The summary of the result is showed in Table 5.1. The R^2 scores are reported for the `cv_set` and the test set generated after `train_test_split`.

Table 5.1: Final Hyperparameters for XGB RANSAC and Model Evaluation Results

Hyperparameters		**Test Results**	
degree	2	R^2 Score Test Set	0.8893
colsample_bytree	0.9274		
learning_rate	0.3028		
max_depth	15		
n_estimators	20		
reg_alpha	0.1242	R^2 Score CV Set	0.620
reg_lambda	0.7352		
subsample	0.6777		
min_samples	0.2392		
residual_threshold	5.0912		
max_trials	136		

5.2 Results of Movement Prediction

The XGB Classifier model run according to section 3.5 showed promising result in being accurate about prediction the movement. The training set contained all the recordings from 1 to 39. The summary of the result is showed in Table 5.2. The accuracy score is reported for the maximum accuracy acheived in the trial conducted by `Optuna Study`. The metric used for this is the `accuracy_score`[25].

Table 5.2: Final Hyperparameters for Movement Prediction XGB Classfier

Hyperparameters		Test Results	
`degree`	1		
`colsample_bytree`	0.9065		
`learning_rate`	0.7544		
`max_depth`	10		
`n_estimators`	10		
`reg_alpha`	2.10847	`Movement Prediction accuracy over the test split`	0.996
`reg_lambda`	9.1995		
`subsample`	0.99949		
`min_samples`	0.7556		
`residual_threshold`	2.3906		
`max_trials`	136		

5.3 Performance of Our Filter on BROAD Dataset Vs Standard Implementations

The experimental setup was implemented following the steps mentioned in subsection 4.3.3. The `cv_set` contains a variation of 5 file indexes of different motions. Thus results were generated based on these 5 files where noise removal was done. Results for simply performing toggling was evaluated over all the 39 recordings.

5.3.1 Results with only Toggling Engine, no noise Removal

In this subsection the charts of Mean errors and Error variance have been shown in Figure 5.1 and Figure 5.2 respectively. These charts were generated from the implementation of algorithm 1 but without the noise prediction model removing the noise from the sensor readings. The duty cycle chosen was 66% and α was chosen a low value of `0.001`.

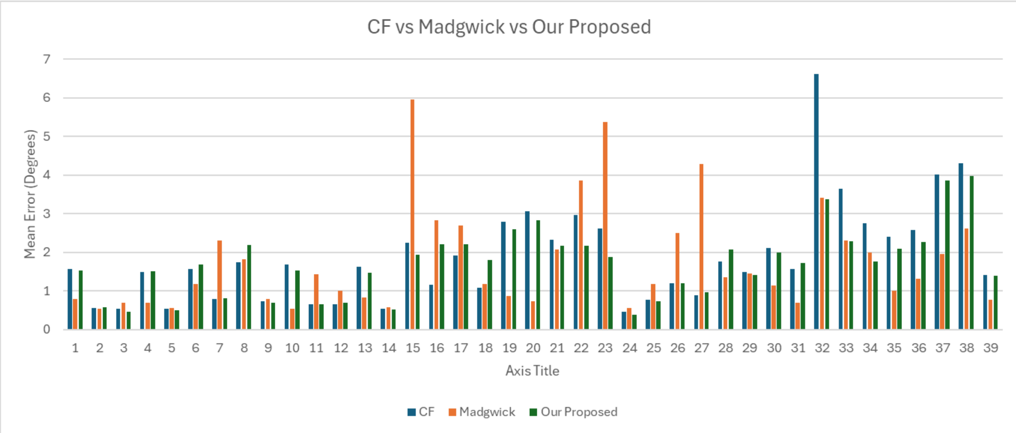


Figure 5.1: Mean of errors on all 39 recordings, for the Madgwick, Complementary , and our Complementary filtering with movement prediction and toggling with 66% duty cycle

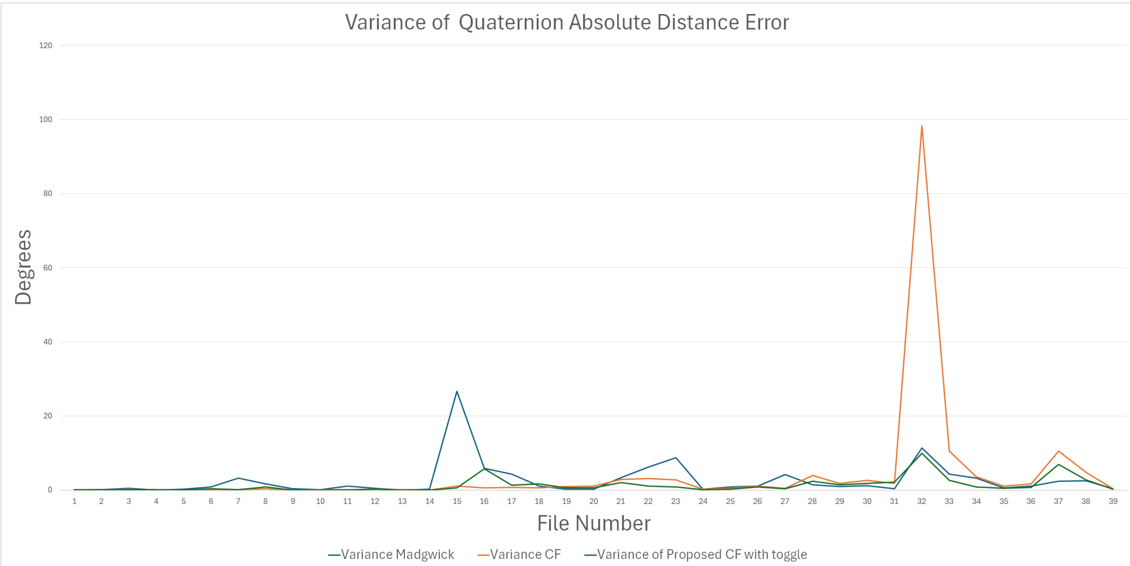


Figure 5.2: Variance of errors on all 39 recordings, for the Madgwick, Complementary , and our Complementary filtering with movement prediction and toggling with 66% duty cycle

5.3.2 Results of CF with α-toggling and Noise removal

This subsection displays the result of α-toggling noise removing Complementary filter which is the complete realization of algorithm 1. The charts only compare the results on 5 files of the `cv_set` containing `[8, 14, 17, 22, 30]`, as the other files were used to train the noise removal model. The mean and variance results are shown in Table 5.3 and Table 5.4 respectively, where lower values denote better performance. The chart in Figure 5.3 shows a comparision between the noise removal approach with other filters.

Table 5.3: Mean Error Statistics across the cv_set

Filename	**Basic CF**	**Proposed CF**	**MadgWick Filter**
09 undisturbed fast rotation with breaks B.hdf5	0.8134	**0.7408**	0.7899
15 undisturbed fast translation A.hdf5	3.1007	**1.6212**	5.9503
18 undisturbed fast translation with breaks B.hdf5	5.5009	1.2757	**1.1728**
23 undisturbed fast combined 360s.hdf5	3.6085	**2.7388**	5.3682
31 disturbed stationary magnet D.hdf5	2.4708	1.5798	**0.6953**

Table 5.4: Variance of Error Statistics across the cv_set

Filename	**Basic CF**	**Proposed CF**	**MadgWick Filter**
09 undisturbed fast rotation with breaks B.hdf5	**0.1110**	0.2603	0.3510
15 undisturbed fast translation A.hdf5	3.8544	**0.9997**	26.7320
18 undisturbed fast translation with breaks B.hdf5	31.1229	1.1998	**1.1499**
23 undisturbed fast combined 360s.hdf5	2.8928	**1.1746**	8.7074
31 disturbed stationary magnet D.hdf5	4.9321	1.5576	**0.3195**

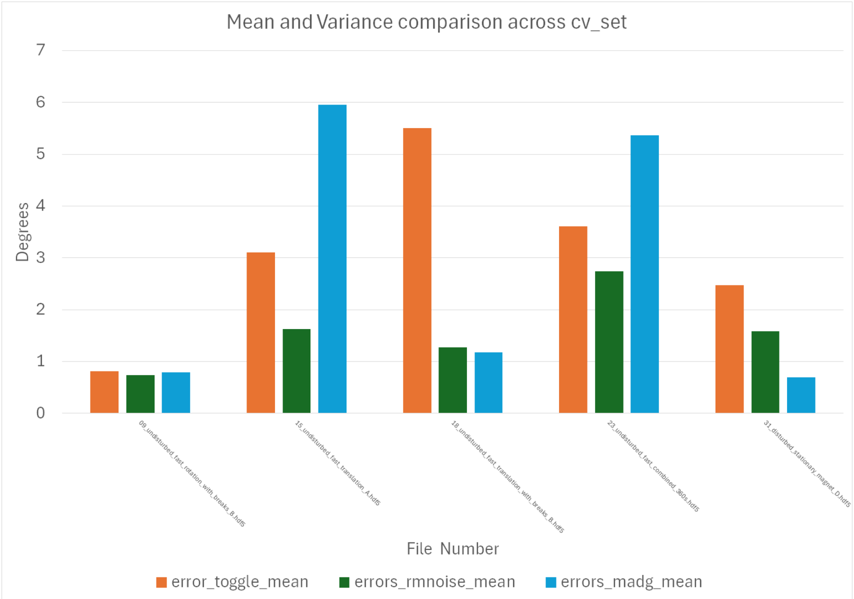


Figure 5.3: Mean and Variance comparison across cv_set. Filenames are sequentially arranged according to Table 5.3

5.3.3 Results of CF with α-Toggling and Noise Removal on Individual Files

This subsection illustrates the performance of the filter on a particular recording of the BROAD dataset over the iterations. Before demonstration of the graphs, the process how the error graphs on individual files have been generated should be stated.

- The individual record files have 50,000 to 300,000 sensor readings. Each reading correspond to a Quaternion Estimate q_{esti}.
- Each reading also corresponds to a ground truth quaternion q_{gti}.
- Their distance is calculated according to the process mentioned in subsection 4.3.3.
- This distance/error is plotted vs the iteration count in figures 5.4,5.5,5.6,5.7 and 5.8.
- **Legend:**
  - ***Red:*** Madgwick Filter
  - ***Green:*** Complementary Filter
  - ***Cyan:*** AQUA Filter
  - ***Blue:*** Our Proposed Filter

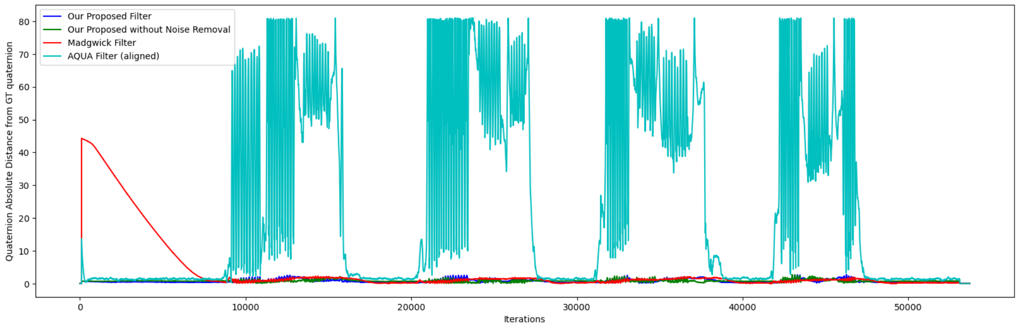


Figure 5.4: Comparison of mean error for 9_undisturbed_fast_rotation_with_breaks_B

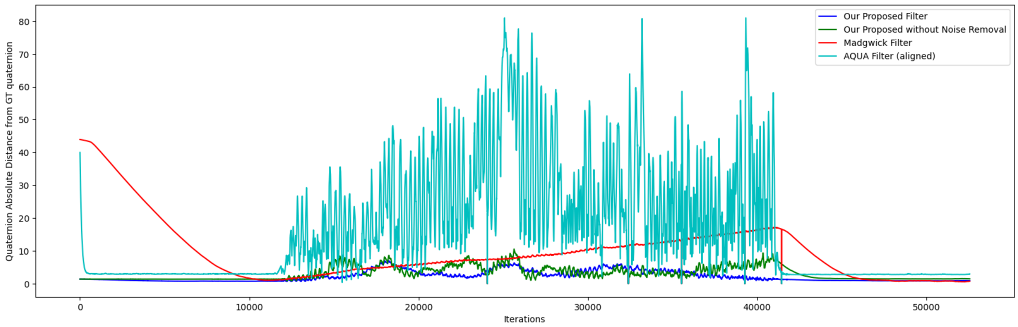


Figure 5.5: Comparison of mean error for 15_undisturbed_fast_translation_A

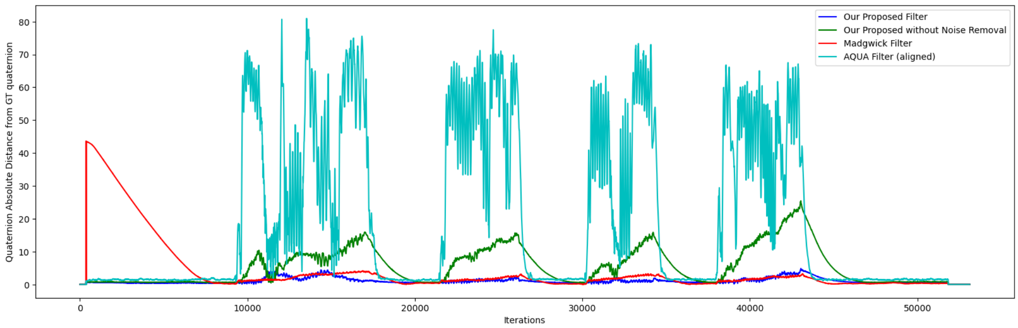


Figure 5.6: Comparison of mean error for 18_undisturbed_fast_translation_with_breaks_B

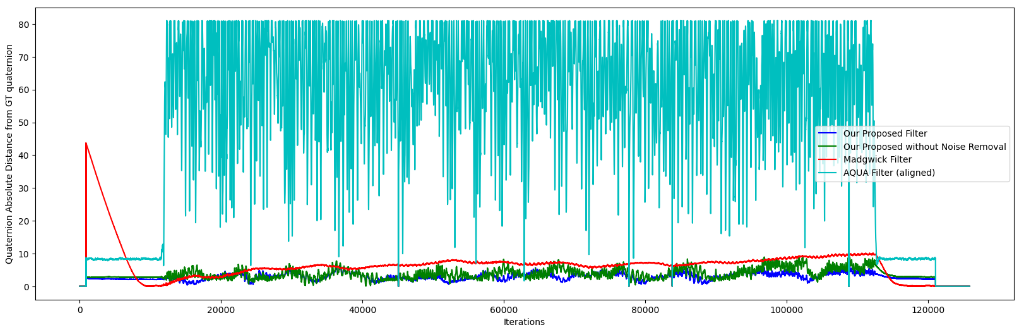


Figure 5.7: Comparison of mean error for 23_undisturbed_fast_combined_360s

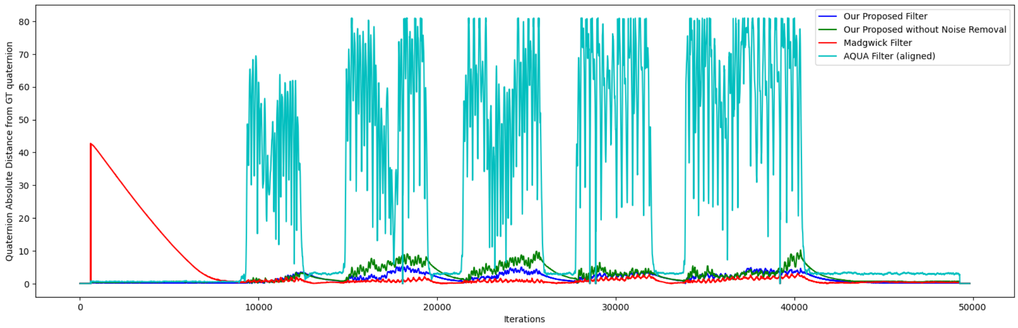


Figure 5.8: Comparison of mean error for 31_disturbed_stationary_magnet_D

5.4 Performance of Proposed Filter against other filters on the Mechanical Platform

Here the results of running our proposed filter is tested along with the performaces of other standard existing filters - *AQUA* [43], *EKF* [34] and the MadgWick Gradient Descent filter [29].

5.4.1 Static Error Statistics

Table 5.5: Static error metrics.

Metric	Proposed (denoising)	Proposed (no denoising)	Madgwick	Aqua	EKF
mean	23.1	14.99	79.71	**10.64**	39.05
std	2.24	**1.06**	8.44	2.16	19.18
min	0.02	0.02	**0.006**	3.28	10.99
25%	22.55	15.05	80.83	**7.51**	22.043
50%	22.73	15.07	80.84	**11.935**	33.717
75%	24.50	15.10	80.85	**2.03**	53.692
max	25.53	15.3	81.03	**13.986**	81.03

5.4.2 Quasi Static Error Statistics

Table 5.6: Quasi Static error metrics (°) for the X-axis.

Metric	Proposed (ML)	Proposed (no ML)	Madgwick	Aqua	EKF
mean	16.25	14.06	13.45	**12.32**	52.278
std	22.04	**21.03**	21.15	23.9	15.32
min	**0.001**	0.002	0.006	0.862	35.4
25%	**0.001**	0.002	0.006	0.862	35.4
50%	6.42	7.32	4.55	**1.36**	64.95
75%	18.84	14.71	15.77	**5.36**	64.88
max	80.98	**80.1**	80.7	81.58	81.02

Table 5.7: Quasi Static error metrics (°) for the Y-axis.

Metric	Proposed (ML)	Proposed (no ML)	Madgwick	Aqua	EKF
mean	18.04	17.36	14.77	**11.44**	40.89
std	23.01	24.10	23.89	23.92	**17.5**
min	0.002	**0.0005**	0.006	0.38	22.77
25%	**0.002**	0.0442	0.006	0.38	22.81
50%	7.15	9.29	2.69	**0.862**	49.79
75%	27.932	25.09	21.59	**4.72**	52.96
max	81.02	81.02	80.97	82.56	**80.9**

Table 5.8: Quasi Static error metrics (°) for the Z-axis.

Metric	Proposed (ML)	Proposed (no ML)	Madgwick	Aqua	EKF
mean	9.84	**7.5**	7.98	10.08	32.2
std	10.33	**7.26**	9.77	18.77	18.122
min	0.02	**0.0029**	0.006	1.81	16.81
25%	0.03	**0.0029**	0.005	1.81	18.61
50%	6.04	7.94	3.22	**1.81**	24.626
75%	19.74	14.06	14.56	**6.16**	42.645
max	36.89	**31.22**	48.39	82.2	80.995

5.4.3 Dynamic Error Statistics

Table 5.9: Dynamic error metrics (°) for the X-axis.

Metric	Proposed (ML)	Proposed (no ML)	Madgwick	Aqua	EKF
mean	30.87	**22.12**	32.35	24.36	47.57
std	31.33	**18.87**	31.86	31.06	19.17
min	**0.0007**	0.001	0.0066	0.883	5.35
25%	0.0435	**0.001**	0.0486	0.883	25.08
50%	15.87	16.08	15.29	**3.57**	50.97
75%	68.16	**41.99**	66.77	50.92	62.07
max	**81.02**	**81.02**	81.03	81.65	81.03

Table 5.10: Dynamic error metrics (°) for the Y-axis.

Metric	Proposed (ML)	Proposed (no ML)	Madgwick	Aqua	EKF
mean	26.92	33.97	29.90	**22.54**	44.08
std	28.90	29.95	30.74	28.58	**24.48**
min	**0.0016**	0.0026	0.0066	1.390	12.27
25%	**0.0016**	0.192	0.0066	1.390	13.13
50%	17.65	30.70	12.98	**2.02**	47.01
75%	**56.71**	63.90	62.82	52.27	67.03
max	81.03	**81.00**	81.02	82.01	**81.00**

Table 5.11: Dynamic error metrics (°) for the Z-axis.

Metric	Proposed (ML)	Proposed (no ML)	Madgwick	Aqua	EKF
mean	29.80	**10.14**	25.41	17.95	30.07
std	28.14	**11.72**	26.86	24.03	17.38
min	0.0023	**0.0004**	0.0065	1.723	11.37
25%	**0.0023**	0.04	0.0065	1.723	17.58
50%	25.17	8.25	13.15	**4.16**	25.59
75%	54.14	**13.33**	52.18	29.84	29.91
max	81.02	**80.66**	81.01	82.25	81.03

5.4.4 Resource Consumption Statistics

Table 5.12: Static Error Resource consumption per filter on RPI: average CPU (%) and memory (MB).

Filter	CPU Usage (%)	Memory Usage (MB)
Proposed (denoising)	58.6	145.65
Proposed (no denoising)	54.5	120.35
Madgwick	1.75	59.75
EKF	3.12	59.77
Aqua	2.32	59.69

Chapter 6

Interpretation of Results

In this chapter, the results achieved in chapter 5 will be interpreted in detail. The results we have acheived so far is promising, and it is able to outperform standard Madgwick filter implementation in fast motion scenarios. However, these promising results come with compromises. The results are promising only in the dataset simulations and also fail to reasonably converge in the tests run on the mechanical platform as shown in section 5.4.

6.1 Results on the BROAD Dataset

6.1.1 Results Without Denoising

In Figure 5.1 and Figure 5.2 we can observe spikes in the orange line graphs which represent the Madgwick Filter. Our proposed filter (in Green) is able to avoid high spikes in files `15_undisturbed_fast_translation_A`, `23_undisturbed_fast_combined_360s` and in `27_disturbed_phone_vibration_B` in comparisions of mean error in Figure 5.1. This indicates that in fast motion scenarios the toggling of α has a positive effect on the error in our proposed filter avoiding higher mean error. In case of variance comparison in Figure 5.2 we see that our proposed method (green line) usually has the lowest variance compared to the other two, but there are a few cases where it does not.

Intuitive Reasoning Backed by Emperical Evidence

Here we suggest why α-toggling might result in a better performance.

- Toggling the α is done in scenarios when there is movement. When the model predicts there is some movement instead of varying the alpha, the alpha is toggled according to defined duty cycle and frequency, this gives just enough information for the gyroscope error to correct itself.

- When there is no movement(predicted by the model), then the α is set to its predefined value. Here the assumption is that the system is much less noisy when there is less movement, and thus can employ more trust to the accelerometer and magnetometer by keeping the alpha to a constant value.

6.1.2 Results With Denoising

The results shown after denoising is much more promising than the results without denoising. In Table 5.3 and Table 5.4 we can observe that our proposed filter performs better than atleast one of the filters in all five trials of the `cv_set`. In case of variance of errors in Table 5.4, our filter consistently is able to maintain error variance around 1.0 where the other compared filters often exceed and reach a variance 10x or 20x of our proposed filter.

From the tables, it is observed that in Table 5.3, the best performance of our filter is visible in entries 2 and 4 which correspond to the files `15_undisturbed_fast_translation_A` and `23_undisturbed_fast_combined_360s` respectively. Thus our filter is able to outperform the standard Madgwick filter implementation in fast motion scenarios.

If we observe the error graphed vs iterations for `15_undisturbed_fast_translation_A` in Figure 5.5 in subsection 5.3.3, we observe that the Madgwick Filter's error steadily rises where our proposed complementary filter stays close to zero. For the file Figure 5.6, we observe that Madgwick filter performs marginally better.

Intuitive Reasoning Based on Emperical Evidence

- As noise is subtracted from the accelerometer and magnetometer readings, the extraneous linear acceleration is subtracted, as a result the orientation estimation is far superior in case of fast motions.

- It is observed from Figure 5.6 that the MadgWick filter stabilizes and converges to a good orientation when there is no motion, thus in the file `18_undisturbed_fast_translation_with_breaks_B` the Madgwick filter performs better than our filter as it has breaks to recover from its error. In case of Figure 5.5 for the file `15_undisturbed_fast_translation_A` the Madgwick filter is not able to stabilize itself,as there are no breaks and hence our filter performs better.

6.2 Results of Tests on the Turntable

The results obtained from experiments on the physical turntable were less promising compared to those generated using the dataset. As discussed in section 5.4, the proposed filter performs comparably to other contemporary orientation filters. However, it exhibits a significantly higher computational cost (see Table 5.12) due to the incorporation of machine learning (ML) based algorithms.

- With the ML module enabled, the filter performs close to the version with ML disabled, though the results indicate it is slightly worse.
- The toggle ML algorithm appears to fail in reliably predicting motion events, which limits its effectiveness in reducing computational overhead.
- Our algorithm performs similar in all the three movement scenarios described in Table 4.1.
- Our proposed filter performs comparatively better with a lower α value.

6.2.1 Results without ML

The tests without the machine learning module provides important insights into the comparative performance of different filters. The results for both with and without ML are summarized in Table 5.5–Table 5.11. The observations can be summarized as follows:

- **Proposed Toggling Engine:**
  - Maintains consistently low error throughout the dynamic rotation sequence in the Z axis.
  - Demonstrates robustness in handling fast rotational changes without large fluctuations.
  - Unfortunately, the X and Y axes fail to show similar results which might be caused due to improper setup.
- **AQUA Filter:**
  - Exhibits high error spikes during dynamic rotational scenarios.

- The spikes indicate sensitivity to rapid orientation changes, reducing reliability in dynamic conditions.

- **Madgwick Filter:**
  - Shows a steady accumulation of error over time rather than sudden spikes.

- **Overall Conclusion:**
  - The toggling engine provides the most balanced performance, avoiding both error spikes (seen in AQUA) and steady drift (seen in Madgwick). But this result is only visible in the Z-axis testing and not in the graphs of X and Y axes. Refer to chapter 10 for the graphs.
  - These results confirm the effectiveness of the toggling mechanism, even without machine learning, in ensuring reliable orientation estimation in dynamic conditions.

6.2.2 Results with ML

The evaluation with the machine learning (ML) based noise removal enabled reveals a different trend compared to the results without ML. The same tables Table 5.5–Table 5.11 also include the ML results. The observations can be summarized as follows:

- **Proposed Filter with ML:**
  - Exhibits high deviation during dynamic motion, with noticeable error spikes.
  - Once the spikes occur, the error often remains elevated instead of returning to baseline levels.
  - This suggests that the ML-based noise removal mechanism may be overcompensating or failing to adapt effectively to certain dynamic conditions.

- **Other Filters (AQUA, EKF, Madgwick):**
  - Show similar performance patterns to those observed without ML in subsection 6.2.1.
  - None of these filters exhibit the persistent deviation spikes that are observed in the ML-enabled proposed filter.
  - Their error remains bounded within expected ranges, although with the same limitations (e.g., AQUA spikes, Madgwick drift) as previously noted.

- **Overall Implication:**
  - While the integration of ML-based noise removal was intended to improve robustness, in practice it introduces instability under dynamic conditions.
  - The behavior indicates a possible mismatch between the training data distribution of the ML model and the real-world data collected from the experimental setup.
  - Further refinement, such as domain adaptation or hybrid approaches, may be necessary to stabilize the ML contribution to the filter.

6.2.3 CPU and Memory Usage

The computational demands of the proposed filter are noticeably higher compared to traditional orientation filters.Summary of the computational statistics is found in Table 5.12. This limitation arises from multiple contributing factors:

- **Impact of Machine Learning Models:** The integration of machine learning models for noise removal significantly increases CPU usage and memory consumption. These models require the storage of multiple decision trees and the traversal of tree structures during inference, which is computationally expensive in comparison to lightweight mathematical filter updates.
- **CPU Usage Without ML:** Even with the ML-based denoising turned off, the CPU utilization remains elevated. This is primarily due to the multithreading mechanisms employed by `NumPy` and other underlying linear algebra libraries (e.g., BLAS, OpenBLAS). These libraries parallelize computations across multiple cores, which, although efficient for large matrix operations, results in unnecessary overhead for smaller real-time filter updates.
- **Memory Overhead:** The storage of ML models (e.g., wide XGBoost trees) contributes to memory complexity. Additionally, intermediate matrices and arrays created during runtime accumulate memory usage, even in scenarios where the ML module is disabled.
- **Future Work:** To address these issues, future development should focus on:
  - Deriving a fully mathematical formulation of the filter that reduces dependency on large external libraries.
  - Implementing optimized linear algebra routines tailored to the filter rather than relying on general-purpose libraries such as `NumPy`.

- Exploring lightweight ML surrogates (e.g., compact neural networks or reduced tree ensembles) if ML-based denoising is to be retained.

Chapter 7

Shortcomings

Our approach is data driven, thus it requires more time and memory complexity to perform inference from our models, which reduces the mathematical simplicity of the complementary filter.

7.1 Algorithmic and Computational Limitations

In addition to the experimental shortcomings, the proposed approach faces several algorithmic and computational challenges, primarily due to its data-driven design and reliance on machine learning models. These limitations are summarized as follows:

- **Size of the Model:** The inference time of tree-based ensemble models, such as XGBoost, is theoretically $O(\log(n))$, where n denotes the number of nodes per tree. In practice, inference complexity is bounded by the maximum tree depth, which was limited to 15 in our implementation. For 6 input variables and 15 estimators, approximately $15 \times 6 \times 6$ decision operations are required per inference step. While this number is not prohibitively large, the memory footprint becomes problematic because XGBoost trees tend to be wide, requiring substantial storage of split thresholds and leaf values [12]. This memory demand could be critical in embedded or resource-constrained environments.

- **Cost of Matrix Exponentiation:** Orientation estimation requires rotation of vectors using rotation matrices derived from quaternions. Efficient computation often involves matrix exponentiation or repeated multiplication, which is computationally expensive, especially when executed in real time on embedded processors. Although various approximations and fast algorithms exist [18], their incorporation requires careful error analysis to ensure that orientation accuracy is not compromised.

- **Training Time:** Training with RANSAC (Random Sample Consensus) significantly increases the overall computational cost. While RANSAC improves robustness to outliers, its iterative nature and repeated model fitting make the training pipeline time-consuming. This limits scalability when experimenting with larger datasets or tuning hyperparameters for better performance.

- **Generalizability:** The trained models demonstrated stronger performance on the test set compared to the cross-validation set, suggesting potential overfitting or dataset bias. This raises concerns about the model's ability to generalize to unseen scenarios or different sensor conditions. Over-reliance on a specific dataset distribution may lead to degraded performance when deployed on live hardware with varying noise, dynamics, or environmental factors [21].

7.2 Experimental Platform and Evaluation

During the evaluation of the orientation filters on the mechanical turntable, several limitations and open issues were identified. These are discussed below:

- **Quaternion Initialization:** The tested filters do not employ a standardized method for initializing the quaternion estimate at startup. Differences in initialization strategies can introduce systematic biases in the early stages of estimation, particularly affecting filters such as the EKF, which rely heavily on linearization around the initial state [34]. A careful analysis of consistent initialization methods is required to reduce the initial error and improve comparability between filters.

- **Mismatch in Training Data for ML Models:** The machine learning models were trained using data generated following the methodologies outlined by the authors of the BROAD dataset [26]. However, discrepancies between this dataset, as well as variations across different sensor units and the live data acquired from the Raspberry Pi–based experimental platform, may have led to poor generalization. Differences in sensor noise characteristics, sampling rates, and motion profiles across datasets and hardware units can result in unreliable inference. Future work should focus on domain adaptation techniques or retraining the models using data collected directly from the experimental hardware. In particular, domain adaptation aims to reduce the performance gap that arises when a model trained on one data source (e.g. simulated or pre-recorded data) is applied to another with different characteristics (e.g. real sensor data from the Raspberry Pi setup).

- **Trust in the AHRS Python Library:** The AHRS Python library [3] was used to implement several of the orientation filters, including AQUA, EKF, and Madgwick. While the library is well-documented and widely adopted for research prototyping, there is a lack of independent validation studies confirming that its implementations are consistent with the original algorithm specifications. Without rigorous benchmarking against reference implementations, there remains uncertainty regarding the correctness of the results produced by these filters.

- **Rigidity of the Absolute Distance Metric:** The evaluation employed the quaternion absolute distance metric to quantify the deviation of estimated orientations from the ground-truth quaternion. While this metric is mathematically sound and commonly used in orientation estimation research [32], it may be overly strict in practice. Small angular discrepancies can result in disproportionately large values under this metric, potentially exaggerating the reported errors across all filters. Alternative evaluation metrics, such as angular difference about specific axes or application-dependent error tolerances, may provide a more nuanced picture of filter performance.

Chapter 8

Future Works

This thesis has explored and implemented data-driven orientation estimation methods by combining machine learning models with complementary filtering techniques. Although the results indicate partial but clear improvements over traditional approaches, several limitations still remain. This chapter discusses the challenges found and suggests possible directions for future research to improve the system's performance, ease of use, and real-world relevance.

8.1 Potential Directions for Enhancement

Some of the limitations discussed in chapter 7 highlight opportunities for further refinement. This section outlines the potential directions for enhancement that could address these shortcomings and improve the overall system performance. In the subsequent section, we propose specific strategies and methodologies to pursue these improvements in future work.

Algorithm and Computational Efficiency: The tree-based models we use, like XGBoost, require significant memory and processing power. This can be a problem, especially on small devices with limited resources. Also, matrix exponentiation used for orientation calculations needs a lot of computation, making real-time operation difficult on embedded platforms.

Training Time and Scalability: Using RANSAC during training helps handle outliers but also increases training time. This slows down scaling to larger datasets or thorough hyperparameter tuning. Additionally, because the models are trained on specific datasets, they might not generalize well to new, unseen data.

Mismatch Between Training and Live Data: The data used to train the models, like the BROAD dataset, differs in noise characteristics and motion from the live data collected during experiments. Such differences can reduce the model's accuracy when used in real-life conditions.

Evaluation Metrics: The quaternion absolute distance used to measure performance might be too strict, exaggerating small orientation errors. More suitable metrics that consider practical application needs should be explored.

8.2 Proposed Future Directions

8.2.1 Improve Computational Efficiency

To better suit embedded and low-power devices, work should focus on reducing memory use and speeding up inference. This can be done by pruning large models, using quantization to lower numerical precision, or applying knowledge distillation to create smaller models that perform similarly. Exploring simpler models like decision trees or linear models, possibly enhanced by smart feature design, could also help. Designing specialized hardware to accelerate machine learning computations could make real-time operation feasible on tiny devices.

8.2.2 Online and Adaptive Learning

Currently, models are trained offline and remain static during deployment. Future research could investigate online learning approaches, where models continuously update in response to incoming sensor data. Such methods would enable the system to adapt to sensor degradation, environmental changes, or novel motion patterns, enhancing robustness and reducing reliance on manual retraining. In particular, exploring adaptive algorithms that adjust model parameters in real time based on sensor inputs could improve long-term performance and maintain accuracy under dynamic conditions. Research could also evaluate strategies for balancing adaptation speed with stability, ensuring reliable learning without introducing instability from noisy or inconsistent data.

8.2.3 Better Noise and Disturbance Models

Although noise has been reduced using regression models, there's still room for improvement, especially for time-dependent noise patterns.

Techniques like recurrent neural networks or probabilistic models may capture temporal correlations better. Domain-specific noise, such as vibrations in drones or magnetic interference in factories, should also be studied. Combining robust statistics and outlier detection with sensor fusion algorithms might reduce unwanted noise even more.

8.2.4 Comprehensive Benchmarking and Evaluation

More testing of orientation algorithms on different datasets, IMU hardware, and sensor setups is needed to fully understand their strengths and weaknesses.

Developing a standard set of performance metrics that cover accuracy, computational load, latency, robustness, and resource use would allow fair comparison between methods and promote best practices.

8.2.5 Real-World Deployment and Testing

The current work mainly tests the method on controlled datasets and setups. Future research should implement these algorithms on real-world devices like smartphones, drones, or wearables. Testing in various real environments will better show how the system handles real-life disturbances, sensor noise, and motion complexity. Long-term trials will reveal drift behavior and system stability over time.

8.2.6 Standardization of Tools and Validation

The AHRS Python library used in this thesis has not been fully validated with respect to reference implementations. Future work should include thorough validation of this and similar libraries. Establishing consistent testing protocols and evaluation standards will enable reliable comparison of orientation estimation methods and advance the research field.

References

[1] "A robust complementary filter approach for attitude estimation of unmanned aerial vehicles using AHRS," in *Proceedings of the 2019 CEAS EuroGNC conference*, CEAS-GNC-2019-036, Milan, Italy, Apr. 2019.

[2] Adafruit Industries, *Adafruit as5600 magnetic angle sensor – stemma qt*, `https://www.adafruit.com/product/6357`, Accessed: 2025-09-26, 2025.

[3] *Ahrs: Attitude and heading reference systems*, `https://ahrs.readthedocs.io/en/latest/`, Accessed: 2025-09-06.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631. DOI: `10.1145/3292500.3330701`

[5] A. Asgharpoor Golroudbari and M. Sabour, *End-to-end deep learning framework for real-time inertial attitude estimation using 6dof imu*, Feb. 2023. DOI: `10.22541/au.167628608.89589390/v1`

[6] I. Y. Bar-Itzhack, "Request - a recursive quest algorithm for sequential attitude determination," *Journal of Guidance, Control, and Dynamics*, vol. 19, no. 5, pp. 1034–1038, 1996. DOI: `10.2514/3.21742` eprint: `https://doi.org/10.2514/3.21742`. [Online]. Available: `https://doi.org/10.2514/3.21742`

[7] P. J. Bickel and K. A. Doksum, *Mathematical statistics: basic ideas and selected topics, volumes I-II package*. Chapman and Hall/CRC, 2015.

[8] H. D. Black, "A passive system for determining the attitude of a satellite," *AIAA journal*, vol. 2, no. 7, pp. 1350–1351, 1964.

[9] M. Bramer, "Avoiding overfitting of decision trees," *Principles of data mining*, pp. 119–134, 2007.

[10] M. Brossard, S. Bonnabel, and A. Barrau, "Denoising imu gyroscopes with deep learning for open-loop attitude estimation," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4796–4803, 2020.

[11] A. Cariow and G. Cariowa, “A hardware-efficient approach to computing the rotation matrix from a quaternion,” *arXiv preprint arXiv:1609.01585*, 2016.

[12] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.

[13] Y. Chen and H. Rong, “A customized extended kalman filter for removing the impact of the magnetometer’s measurements on inclination determination,” *Sensors*, vol. 23, no. 24, 2023, ISSN: 1424-8220. DOI: 10.3390/s23249756 [Online]. Available: https://www.mdpi.com/1424-8220/23/24/9756

[14] D. Duong Quoc, J. Sun, V. Le, and L. Luo, “Complementary filter performance enhancement through filter gain,” *International Journal of Signal Processing, Image Processing and Pattern Recognition*, vol. 8, pp. 97–110, Jul. 2015. DOI: 10.14257/ijsip.2015.8.7.10

[15] M. Euston, P. Coote, R. Mahony, J. Kim, and T. Hamel, “A complementary filter for attitude estimation of a fixed-wing uav,” in *2008 IEEE/RSJ international conference on intelligent robots and systems*, IEEE, 2008, pp. 340–345.

[16] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981. DOI: 10.1145/358669.358692

[17] J. R. Forbes, “Fundamentals of spacecraft attitude determination and control [bookshelf],” *IEEE Control Systems Magazine*, vol. 35, no. 4, pp. 56–58, 2015.

[18] G. Gallego and A. Yezzi, “A compact formula for the derivative of a 3-d rotation in exponential coordinates,” *Journal of Mathematical Imaging and Vision*, vol. 51, no. 3, pp. 378–388, 2015.

[19] *Self Localization Using a 9DOF IMU Sensor With a Directional Cosine Matrix*, vol. Volume 4: 18th Design for Manufacturing and the Life Cycle Conference; 2013 ASME/IEEE International Conference on Mechatronic and Embedded Systems and Applications, International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, Aug. 2013, V004T08A051. DOI: 10.1115/DETC2013-12906 eprint: https://asmedigitalcollection.asme.org/IDETC-CIE/proceedings-pdf/IDETC-CIE2013/55911/V004T08A051/4257997/v004t08a051-detc2013-12906.pdf. [Online]. Available: https://doi.org/10.1115/DETC2013-12906

[20] A. Godwin, M. Agnew, and J. Stevenson, “Accuracy of inertial motion sensors in static, quasistatic, and complex dynamic motion,” *Journal of Biomechanical Engineering*, vol. 131, no. 11, p. 114 501, 2009. DOI: 10.1115/1.4000104

[21] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[22] W. T. Higgins, "A comparison of complementary and kalman filtering," *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-11, no. 3, pp. 321–325, 1975. DOI: 10.1109/TAES.1975.308081

[23] D. Q. Huynh, "Metrics for 3d rotations: Comparison and analysis," *Journal of Mathematical Imaging and Vision*, vol. 35, pp. 155–164, 2009.

[24] G. Ke et al., "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.

[25] V. Labatut and H. Cherifi, "Accuracy measures for the comparison of classifiers," *arXiv preprint arXiv:1207.3790*, 2012.

[26] D. Laidig, M. Caruso, A. Cereatti, and T. Seel, "Broad—a benchmark for robust inertial orientation estimation," *Data*, vol. 6, no. 7, p. 72, 2021.

[27] U. López, L. Trujillo, Y. Martinez, P. Legrand, E. Naredo, and S. Silva, "Ransac-gp: Dealing with outliers in symbolic regression with genetic programming," in *Genetic Programming*, Springer, Cham, 2017, pp. 114–130. DOI: 10.1007/978-3-319-55696-3_8

[28] S. O. H. Madgwick, S. Wilson, R. Turk, J. Burridge, C. Kapatos, and R. Vaidyanathan, "An extended complementary filter for full-body marg orientation estimation," *IEEE/ASME Transactions on Mechatronics*, vol. 25, no. 4, pp. 2054–2064, 2020. DOI: 10.1109/TMECH.2020.2992296

[29] S. O. Madgwick, A. J. Harrison, and R. Vaidyanathan, "Estimation of imu and marg orientation using a gradient descent algorithm," in *2011 IEEE international conference on rehabilitation robotics*, Ieee, 2011, pp. 1–7.

[30] R. Mahony, T. Hamel, and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Transactions on automatic control*, vol. 53, no. 5, pp. 1203–1218, 2008.

[31] J. Marins, X. Yun, E. Bachmann, R. McGhee, and M. Zyda, "An extended kalman filter for quaternion-based orientation estimation using marg sensors," in *Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the the Next Millennium (Cat. No.01CH37180)*, vol. 4, 2001, 2003–2011 vol.4. DOI: 10.1109/IROS.2001.976367

[32] F. L. Markley and D. Mortari, "Quaternion attitude estimation using vector observations," *Journal of the Astronautical Sciences*, vol. 48, no. 2–3, pp. 359–380, 2007.

[33] D. Maton et al., "Indirect tuning of a complementary orientation filter using velocity data and a genetic algorithm," *Systems Science & Control Engineering*, vol. 12, no. 1, p. 2 343 303, 2024. DOI: 10.1080/21642583.2024.2343303 eprint: https://doi.org/10.1080/21642583.2024.2343303. [Online]. Available: https://doi.org/10.1080/21642583.2024.2343303

[34] P. S. Maybeck, *Stochastic Models, Estimation, and Control: Volume 1*. Academic Press, 1982.

[35] A. Nawrat, K. Jedrasiak, K. Daniec, and R. Koteras, "Inertial navigation systems and its practical applications," *New approach of indoor and outdoor localization systems*, 2012.

[36] A. Sabatini, "Quaternion-based extended kalman filter for determining orientation by inertial and magnetic sensing," *IEEE Transactions on Biomedical Engineering*, vol. 53, no. 7, pp. 1346–1356, 2006. DOI: 10.1109/TBME.2006.875664

[37] W. Shao, J. Zang, J. Zhao, and K. Liu, "A variable gain complementary filtering fusion algorithm based on distributed inertial network and flush air data sensing," *Applied Sciences*, vol. 13, no. 14, 2023, ISSN: 2076-3417. DOI: 10.3390/app13148090 [Online]. Available: https://www.mdpi.com/2076-3417/13/14/8090

[38] M. K. Al-Sharman, Y. Zweiri, M. A. K. Jaradat, R. Al-Husari, D. Gan, and L. D. Seneviratne, "Deep-learning-based neural network training for state estimation enhancement: Application to attitude estimation," *IEEE Transactions on Instrumentation and Measurement*, vol. 69, no. 1, pp. 24–34, 2020. DOI: 10.1109/TIM.2019.2895495

[39] N. El-Sheimy and A. Youssef, "Inertial sensors technologies for navigation applications: State of the art and future trends," *Satellite navigation*, vol. 1, no. 1, p. 2, 2020.

[40] M. D. SHUSTER and S. D. OH, "Three-axis attitude determination from vector observations," *Journal of Guidance and Control*, vol. 4, no. 1, pp. 70–77, 1981. DOI: 10.2514/3.19717 eprint: https://doi.org/10.2514/3.19717. [Online]. Available: https://doi.org/10.2514/3.19717

[41] M. D. Shuster et al., "A survey of attitude representations," *Navigation*, vol. 8, no. 9, pp. 439–517, 1993.

[42] D. Tedaldi, A. Pretto, and E. Menegatti, "A robust and easy to implement method for imu calibration without external equipments," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 3042–3049. DOI: 10.1109/ICRA.2014.6907297

[43] R. G. Valenti, I. Dryanovski, and J. Xiao, "Keeping a good attitude: A quaternion-based orientation filter for imus and margs," *Sensors*, vol. 15, no. 8, pp. 19 302–19 330, 2015.

[44] E. Vertzberger and I. Klein, "Adaptive attitude estimation using a hybrid model-learning approach," *IEEE Transactions on Instrumentation and Measurement*, vol. 71, pp. 1–9, 2022.

[45] M. Vezočnik, R. Kamnik, and M. B. Juric, "Inertial sensor-based step length estimation model by means of principal component analysis," *Sensors*, vol. 21, no. 10, 2021, ISSN: 1424-8220. DOI: 10.3390/s21103527 [Online]. Available: https://www.mdpi.com/1424-8220/21/10/3527

[46] G. Wahba, "A least squares estimate of satellite attitude," *SIAM Review*, vol. 7, no. 3, pp. 409–409, 1965. DOI: 10.1137/1007077 eprint: https://doi.org/10.1137/1007077. [Online]. Available: https://doi.org/10.1137/1007077

[47] E. Wan and R. Van Der Merwe, "The unscented kalman filter for nonlinear estimation," in *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373)*, 2000, pp. 153–158. DOI: 10.1109/ASSPCC.2000.882463

[48] D. Weber, C. Gühmann, and T. Seel, "Neural networks versus conventional filters for inertial-sensor-based attitude estimation," in *2020 IEEE 23rd International Conference on Information Fusion (FUSION)*, 2020, pp. 1–8. DOI: 10.23919/FUSION45008.2020.9190634

[49] S. Wilson et al., "Formulation of a new gradient descent marg orientation algorithm: Case study on robot teleoperation," *Mechanical Systems and Signal Processing*, vol. 130, pp. 183–200, 2019, ISSN: 0888-3270. DOI: https://doi.org/10.1016/j.ymssp.2019.04.064 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0888327019303012

[50] S. Zhao, "Time derivative of rotation matrices: A tutorial," *arXiv preprint arXiv:1609.06088*, 2016.

[51] Z. Zhe, W. Jian-bin, S. Bo, and T. Guo-feng, "Adaptive complementary filtering algorithm for imu based on mems," in *2020 Chinese Control And Decision Conference (CCDC)*, 2020, pp. 5409–5416. DOI: 10.1109/CCDC49329.2020.9164809

Chapter 9

Quaternion

9.1 Quaternion Fundamentals

9.1.1 Basic Operations with Quaternions

A quaternion q is a four-dimensional number composed of a scalar part and a three-dimensional vector part:

$$q = w + xi + yj + zk$$

where $w, x, y, z \in \mathbb{R}$, and i, j, k are imaginary units satisfying:

$$i^2 = j^2 = k^2 = ijk = -1$$

The basic operations on quaternions are:

- **Addition:** Given two quaternions $q_1 = (w_1, \mathbf{v}_1)$ and $q_2 = (w_2, \mathbf{v}_2)$, their sum is

  $$q_1 + q_2 = (w_1 + w_2, \mathbf{v}_1 + \mathbf{v}_2)$$

- **Multiplication:** The product $q = q_1 \otimes q_2$ is defined as

  $$q = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2,\ w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

  where $\cdot$ denotes the dot product and $\times$ denotes the cross product.

- **Conjugate:** The conjugate of $q = (w, \mathbf{v})$ is

  $$q^* = (w, -\mathbf{v})$$

- **Norm:** The norm of q is given by

$$||q|| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

- **Inverse:** For a nonzero quaternion,

$$q^{-1} = \frac{q^*}{||q||^2}$$

9.1.2 Rotating a Vector Using a Quaternion

To rotate a 3D vector $\mathbf{v} \in \mathbb{R}^3$ using a unit quaternion q, the following procedure is used:

1. Represent $\mathbf{v}$ as a pure quaternion:

$$v_q = (0, \mathbf{v})$$

2. Apply the rotation:

$$v_{\text{rotated}} = q \otimes v_q \otimes q^*$$

   where q^* is the conjugate of q, and $\otimes$ denotes quaternion multiplication.

3. Extract the vector part of v_{rotated} to obtain the rotated vector.

The quaternion rotation formula ensures a smooth and gimbal-lock-free rotation, which is particularly advantageous in 3D applications such as orientation tracking and robotics.

9.2 Absolute Distance

In this section, we describe the computation of the *Quaternion Absolute Distance* (QAD), a common metric used to evaluate differences between two orientation quaternions.

Given two unit quaternions:

- $\mathbf{q}_1 = [w_1, x_1, y_1, z_1]$
- $\mathbf{q}_2 = [w_2, x_2, y_2, z_2]$

The absolute distance $Quat_{abs\_dist}$ between them is defined as:

$$Quat_{abs\_dist}(\mathbf{q}_1, \mathbf{q}_2) = 1 - |\langle \mathbf{q}_1, \mathbf{q}_2 \rangle| = 1 - |w_1 w_2 + x_1 x_2 + y_1 y_2 + z_1 z_2| \tag{9.1}$$

This distance is derived from the dot product between the two quaternions and captures the angular difference in orientation. Since quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation, taking the absolute value of the dot product ensures the distance is invariant to this ambiguity.

9.2.1 Properties

- $Quat_{abs\_dist}(\mathbf{q}_1, \mathbf{q}_2) = 0$ if $\mathbf{q}_1 = \mathbf{q}_2$ or $\mathbf{q}_1 = -\mathbf{q}_2$
- $Quat_{abs\_dist} \in [0, 1]$ for normalized quaternions
- The closer the value is to 0, the more similar the orientations

9.2.2 Use Cases

Quaternion Absolute Distance is frequently used in:

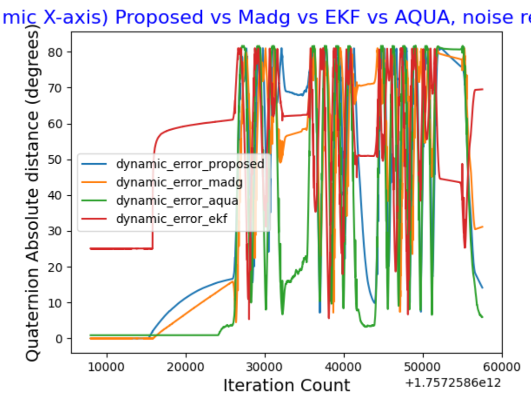
- Orientation estimation
- Motion capture and sensor fusion
- Evaluating the accuracy of inertial tracking algorithms

Chapter 10

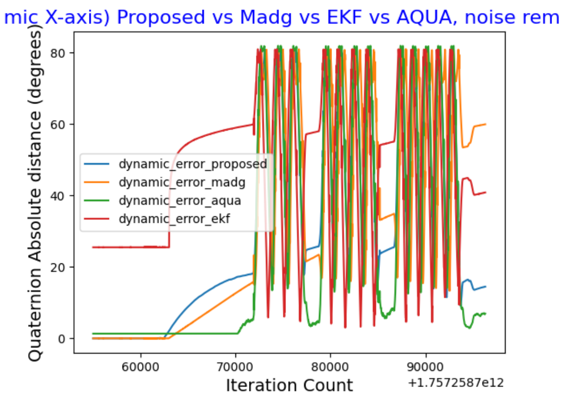
Graphs

10.1 Different Types of Errors

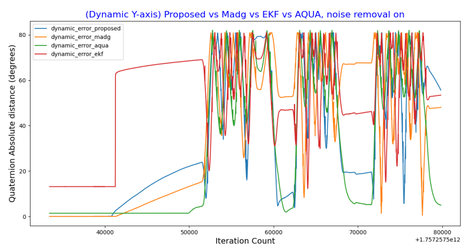
10.1.1 Dynamic Error



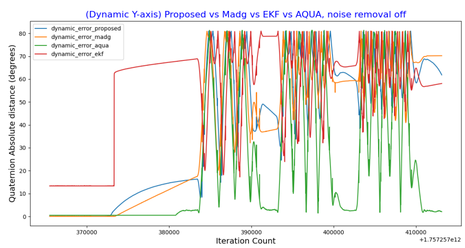
X-axis



X-axis



Y-axis

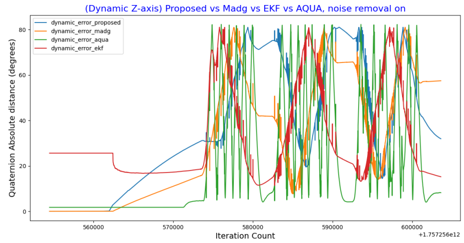


Y-axis

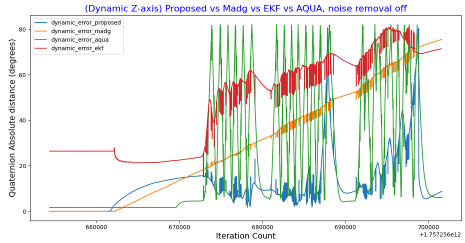
Dynamic Errors with Denoising

Dynamic Errors without Denoising

Figure 10.1: Dynamic Errors (X and Y axes) with and without denoising



Z-axis



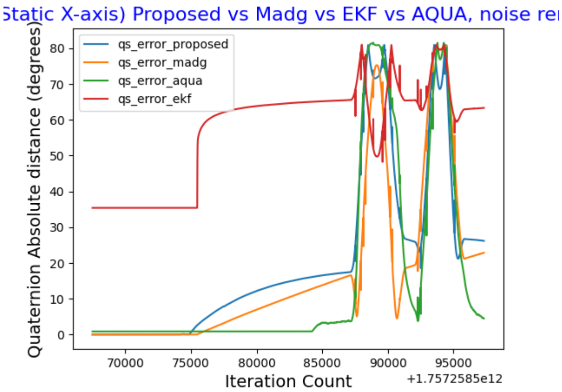
Z-axis

Dynamic Errors with Denoising

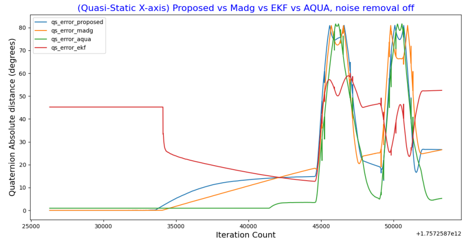
Dynamic Errors without Denoising

Figure 10.2: Dynamic Errors (Z axis) with and without denoising

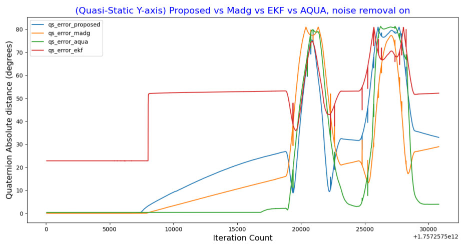
10.1.2 Quasi-static Error



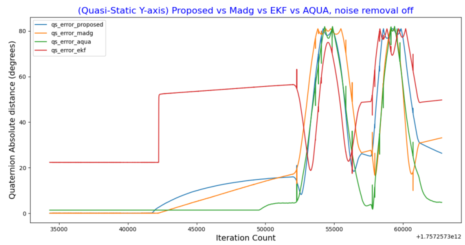
X-axis



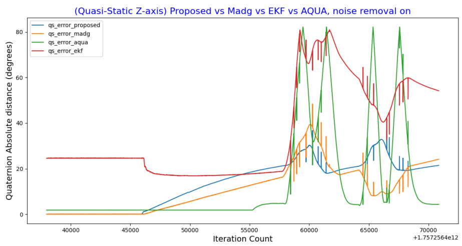
X-axis



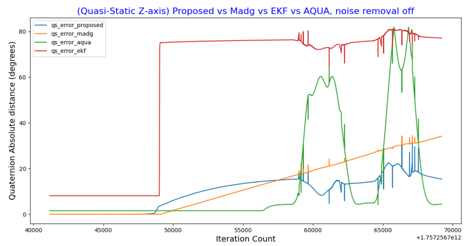
Y-axis



Y-axis



Z-axis



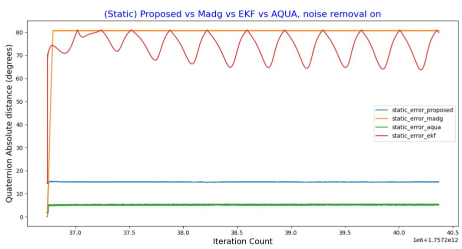
Z-axis

Quasi-static Errors with Denoising

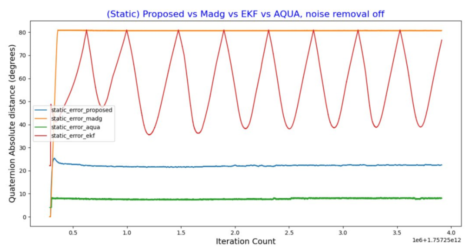
Quasi-static Errors without Denoising

Figure 10.3: Quasi-static Errors with and without denoising

10.1.3 Static Error



Static Error with Denoising



Static Error without Denoising